

CH2、ALU

ALU 的介紹及計算機算術

重點一：數的表示

不同進位法的轉換

$$275_{10} = 2 \times 10^2 + 7 \times 10^1 + 5 \times 10^0 = 4 \times 8^2 + 2 \times 8^1 + 3 \times 8^0 = 423_8 = 100010011_2$$

轉換方式可透過短除法，逆向得到結果

$$\begin{array}{r} 8 \overline{) 275} \quad 3 \\ \underline{240} \\ 35 \quad 2 \\ \underline{32} \\ 3 \quad 4 \\ \underline{32} \\ 0 \end{array}$$

三種數值的表示法：Sign & Magnitude、1's complement、2's complement

Sign & Magnitude：一個位元表示正負值、其餘表示(絕對值)大小，其 3 個缺點如下：

1. 符號位元要置於左或右？
2. 由於我們無法預先知道運算結果的正負，所以在執行加法運算時還需要一個額外步驟來設定符號位元
3. 0 有+0、-0 兩種表示，會導致粗心的程式設計人員發生錯誤

1's complement：以與正數 0、1 相反，來表示負號數值，其 2 個缺點如下：

1. 0 有+0、-0 兩種表示
2. 運算後有時要加 1 做『端回進位』的修正，例如『正大負小』或『兩負相加』：

$$3 + (-1) = 0011 + 1110 = 0001 = 1, \text{ 需端回} +1 \Rightarrow 0001 = 0010 = 2 -$$

$$(-3) + (-1) = 1100 + 1110 = 1010 = -(0101) = -3, \text{ 需端回} +1 \Rightarrow 1011 = -(0100) = -4$$

2's complement：目前電腦採用之方法，為 1's complement 再+1 之表示法

10 到 2 的轉換方式：由右向左、碰到的第一個 1 保留，其左方所有數值 0、1 相反

$$ex : (-3) = -(0011) = 1100 + 1 = 1101$$

2 到 10 的轉換方式：MSB 為負，其餘為正相加

$$ex : 1101 = (-1) \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

	ScM	1's	2's
000	+0	+0	+0
001	+1	+1	+1
010	+2	+2	+2
011	+3	+3	+3
100	-0	-3	-4
101	-1	-2	-3
110	-2	-1	-2
111	-3	-0	-1

例(43) : Number representation.

1. What range of integer number can be represented by 16-bit 2's complement number?
2. Perform the following 8-bit 2's complement number operation and check whether arithmetic overflow occurs. Check your answer by converting to decimal sign-and-magnitude representation.

$$11010011 - 11101100$$

1. -2^{15} to $2^{15}-1$
2. $11010011 - 11101100 = 11010011 + 00010100 = 11100111$

$$\text{Check } -45 - (-20) = -25$$

The range for 8-bit 2's complement number is: -2^7 to $(2^7 - 1)$. So, no overflow.

例(69) : Explain why most of today's computer system use 2's complement instead of signed-magnitude in their hardware implementations.

Signed-magnitude 有以下缺點 :

1. Sign bit 要放於 magnitude 前或後該如何決定 ?
2. 執行加法時需要一額外步驟來決定運算結果為正或負
3. 對 0 的表示方式有正負之分，對於粗心的 programmer 而言，容易導致程式執行時的錯誤

練習 : The following table shows pairs of octal numbers.

	A	B
a.	5323	2275
b.	0147	3457

1. What is the sum of A and B if they represent unsigned 12-bit octal numbers? The result should be written in octal. Show your work.
2. What is the sum of A and B if they represent signed 12-bit octal numbers stored in sign-magnitude format? The result should be written in octal. Show your work.
3. Convert A into a decimal number, assuming it is unsigned. Repeat assuming it stored in sign-magnitude format. Show your work.

	1	2	3
a.	7620	752	2771 -723
b.	3626	3626	103 103

註 : signed 12 bits octal number $5323_8 = 101011010011_2 = -1323_8$

重點二：加法與減法

溢位：正加正、負加負，超過硬體可表示的數值

$$0110(+6) + 0101(+5) = 1011(-5)$$

$$1010(-6) + 1011(-5) = 0101(+5)$$

例(46) : When performing arithmetic addition and subtraction, overflow might occur. Fill in the blanks in the following table of overflow conditions for addition and subtraction.

Operation	Operand A	Operand B	Result indicating overflow
$A+B$	≥ 0	≥ 0	
$A+B$	> 0	< 0	
$A-B$	≤ 0	< 0	
$A-B$	< 0	≥ 0	

Operation	Operand A	Operand B	Result indicating overflow
A+B	≥ 0	≥ 0	< 0
A+B	> 0	< 0	≥ 0
A-B	≤ 0	< 0	< 0
A-B	< 0	≥ 0	≥ 0

例(30) : Determine if each of the following statements is true(T) and false(F).

1. In the IEEE 754 floating-point representation, the precision of represented numbers is determined by the size of fraction.
2. In computer arithmetic, overflow occurs when adding two positive numbers and the sum is negative.

True 、 True

例(47) : Answer the following questions briefly.

1. Will addition "0010 + 1110" cause an overflow using the 4-bit two's complement signed-integer format? (Simply answer yes or no).
2. What would you get after performing arithmetic right shift by one bit on 1100?
3. If one wishes to increase the accuracy of the floating-point numbers that can be represented, then he/she should increase the size of which part in the floating-point format?
4. Name one event other than branches or jumps that the normal flow of instruction execution will be changed, ex: by switching to a routine in the operating system.

1.No 、 2.1101 、 3.Fraction 、 4.Arithmetic overflow

練習、例(第一章 55) : Some programming languages allow 2's complement integer arithmetic on variables declared byte and half. What MIPS instructions would be used?

1. Load with lbu, lhu; arithmetic with add, sub, mult, div; then store using sb, sh.
2. Load with lb, lh; arithmetic with add, sub, mult, div; then store using sb, sh.
3. Load with lb, lh; arithmetic with add, sub, mult, div; then store using and to mask result to 8 or 16 bits after each operation; then store using sb, sh.

2

練習 : Consider 2's complement 4-bit signed integer addition and subtraction. Since the operands can be negative or positive and the operator can be subtraction or addition, there are 8 possible combinations of inputs. For each of them, describe how the overflow can be computed from the sign of the input operands and the carry out and sign of the output.

Sign(Input 1)	Sign(Input 2)	Operation	Sign(Output)	Overflow(Y/N)
+	+	+	+	N
+	+	+	-	Y
+	+	-	+	N
+	+	-	-	N
+	-	+	+	N
+	-	+	-	N
+	-	-	+	N
+	-	-	-	Y
-	+	+	+	N
-	+	+	-	N
-	+	-	+	Y
-	+	-	-	N
-	-	+	+	Y
-	-	+	-	N
-	-	-	+	N
-	-	-	-	N

練習：Find the shortest sequence of MIPS instructions to determine the absolute value of a two's complement integer. Convert this instruction (accepted by the MIPS assembler):

Abs \$t2, \$t3

This instruction means that register \$t2 has a copy of register \$t3 if register \$t3 is positive, and the two's complement of register \$t3 if \$t3 is negative. (Hint: It can be done with three instructions.)

```
addu $t2, $zero, $t3
bgez $t3, Next
sub $t2, $zero, $t3
```

Next:

練習：

a.	\$s0 = 0x7000 0000	\$s1 = 0xFFFF FFFF
b.	\$s0 = 0x4000 0000	\$s1 = 0x4000 0000

- For the contents of register \$s0 and \$s1 as specified above, what is the value of \$t0 for the following assembly code:

```
add $t0, $s0, $s1
```

Is the result in \$t0 the desired result, or has there been overflow?

- For the contents of register \$s0 and \$s1 as specified above, what is the value of \$t0 for the following assembly code:

```
sub $t0, $s0, $s1
```

Is the result in \$t0 the desired result, or has there been overflow?

- For the contents of register \$s0 and \$s1 as specified above, what is the value of \$t0 for the following assembly code:

```
add $t0, $s0, $s1
```

```
add $t0, $t0, $s0
```

Is the result in \$t0 the desired result, or has there been overflow?

	1	2	3
a.	7FFF FFFF, no overflow	6000 0001, no overflow	FFFF FFFF, overflow
b.	8000 0000, overflow	0, no overflow	C000 0000, overflow

重點三：多媒體算數

許多圖學系統使用 3 個 8 bits/1 byte (R、G、B 三原色，0-255，全彩/True color)，來記錄一個點的顏色；用另一個 8 bits/1 byte (alpha)，來記錄位置。但是在運算上，一次只會有 1 byte 的資料量(單一顏色)，若是以傳統 1 個 word(4 bytes/32 bits)的方式運算，過於浪費空間，因此(現代計算機)以 64 位元加法器中的進位鏈分割開來，處理器一次可處理 8 組 8 bits 運算元、或四組 16 bits 運算元、或 2 組 32 bits 運算元的短向量進行運算(若是 MIPS 的話，一次 ALU 為 32 bits，故可處理 4 組 8 bits 資料量)，此種運算稱為 Vector 或 SIMD

而若有超出或低於範圍的計算結果出現時，以頂數或底數來代替之(ex : 0 或 255)，此種方式稱為『飽和運算 Saturating operations』

練習：

- Assume A and B are signed 8-bit decimal integers stored in two's-complement format. Calculate A+B using saturating arithmetic. The result should be written in decimal. Show your work.

- Assume A and B are signed 8-bit decimal integers stored in two's-complement format. Calculate A-B using saturating arithmetic. The result should be written in decimal. Show your work.
- Assume A and B are signed 8-bit integers. Calculate A+B using saturating arithmetic. The result should be written in decimal. Show your work.

	1	2	3
a.	$-56+103=47$	$-56-103=-128(-159)$	$200+103=255(303)$
b.	$-9-19=-28$	$-9+19=10$	$247+237=255(484)$

註：8-bit 2's complement integers $200_{10} = 11001000_2 = -128+64+8 = -56_{10}$

例(26)：Given a finite word-length size, overflow occurs when a result is too large to be represented accurately; however, underflow occurs when a number is too small to be represented correctly. The table below shows pairs of decimal numbers.

- Assume A and B are signed 8-bit decimal integers stored in 2's complement format. Calculate A+B and A-B using saturating arithmetic for the two cases. The result should be written in decimal. Show your work. Is there overflow, underflow, or neither?
- Assume A and B are signed 8-bit integers. Calculate A+B and A-B using saturating arithmetic for the two cases. The result should be written in decimal. Show your work. Is there overflow, underflow, or neither?

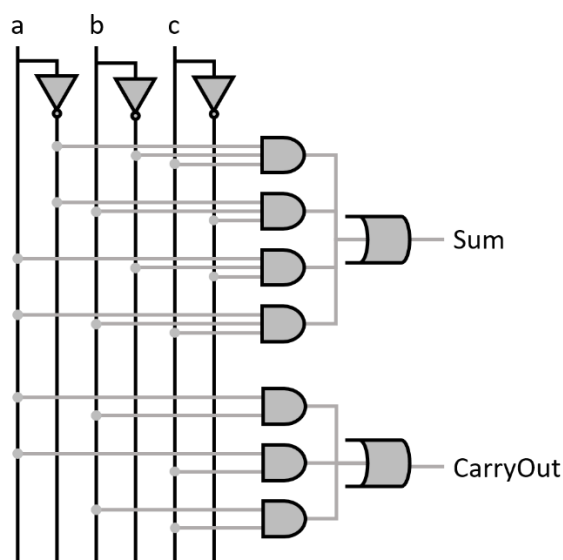
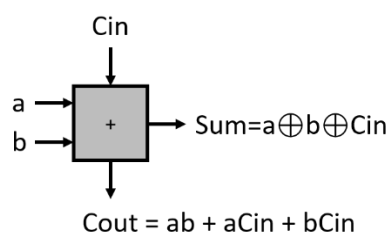
	A	B
case a	200	103
case b	247	237

		1	2
case a	A+B	$-56+103=47$	$200+103=255$
	A-B	$-56-103=-128$	$200-103=97$
case b	A+B	$-9-19=-28$	$247+237=255$
	A-B	$-9+19=10$	$247-237=10$

重點四：算數邏輯運算單元

加法器

Input			Output	
a	b	Cin	Cout	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

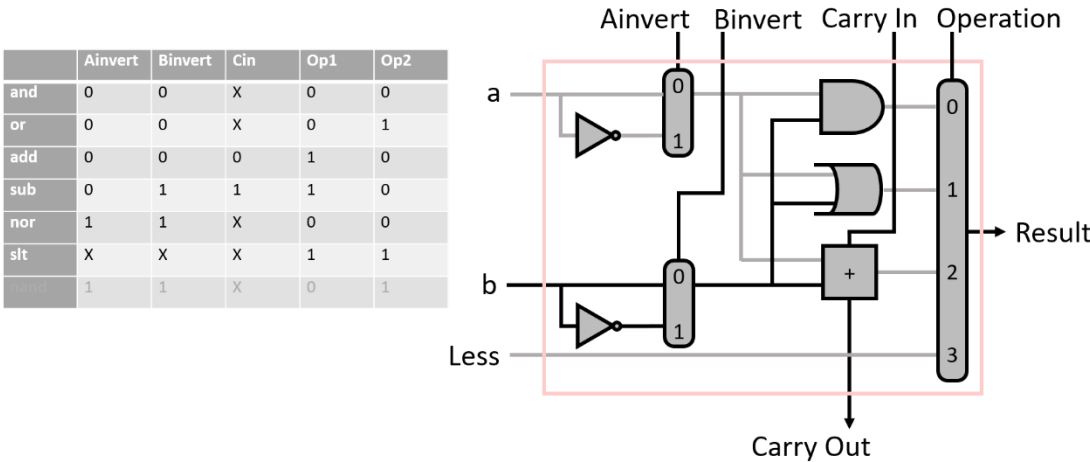


首先求得 a、b、Cin 的運算結果 Sum 與 Cin，再利用 k-map 或消去法...等各種方式導出關係式，最後再以硬體 and、or、not 等邏輯閘，設計出硬體

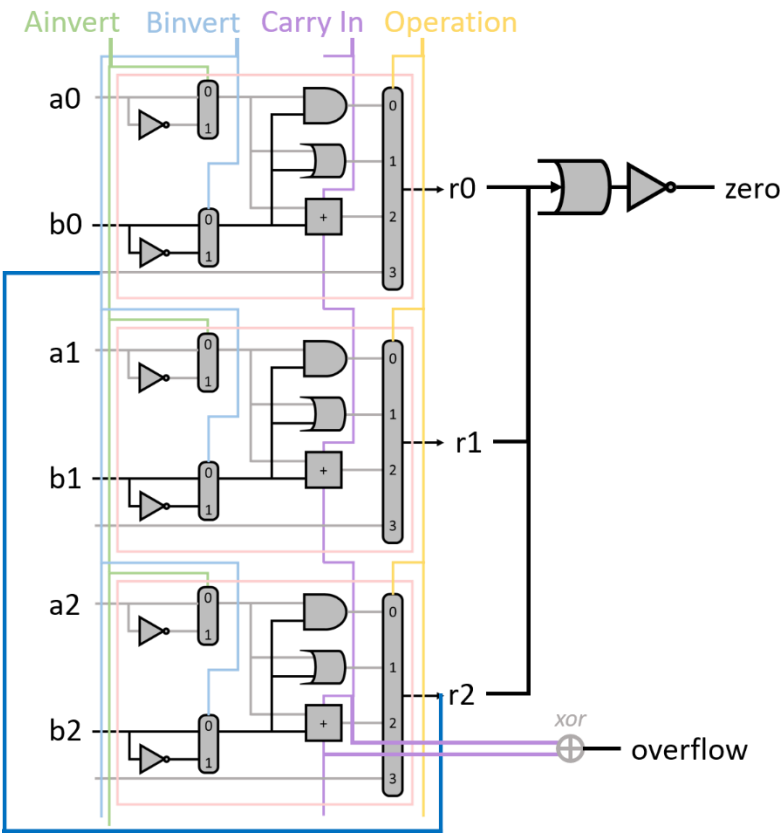
而我們也可以透過上圖發現，Sum 最多會需要經過 3 個邏輯閘、Carry Out 則為 2 個，故時間差上會分別有 3 個 gate delay 與 2 個 gate delay

基本 ALU

又因為 MIPS 提供多種運算元，只需要簡單加入一些硬體，便可執行多種運算 (add、or、add、sub、nor、slt，MIPS 雖然沒有直接提供 nand 的運算，但亦可以透過此 ALU 來完成)



多 bits ALU(32 bits)



Zero：將每一個小 ALU 的結果都 or 起來再 invert，即可知是否為 zero

Overflow：xor 最大的兩個進位位數，便可以知道是否有溢位

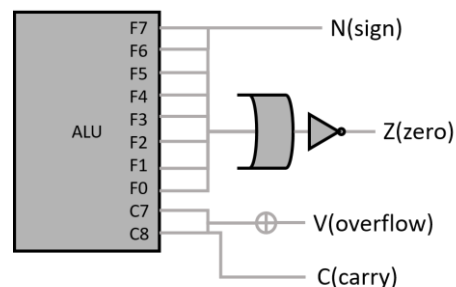
例(33)：Design a one-bit ALU that can support AND/NAND, OR/NOR, ADD and 2's complement subtraction. Describe what should be added to make it work as the MSB and LSB, if we need to construct a multi-bit ALU.

去掉 less 的 ALU，見上圖

例(77)：Design a 8-bit ALU that can perform AND, OR, XOR, addition, and subtraction.

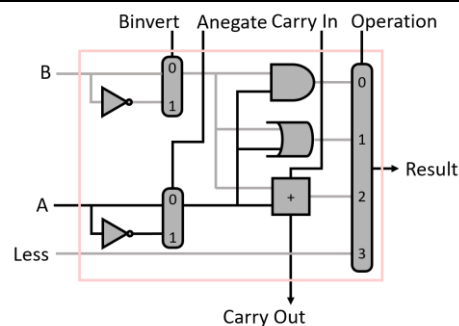
去掉 less 的 ALU，見上圖

例(13)：Given an 8-bit ALU with outputs F7 THROUGH f0 AND AVAILABLE CARRIES c8 AND c7, show the logic circuit for generating the signals for the four status bits N(sign), Z(zero), V(overflow), and C(Carry).



例(21)：The following figure shows an implementation of ALU. Please give the 4-bit control value for the following operations. Fill out the blanks.

Operations	4 bit (Anegate, Binvert, ALUop)
B sub A	
A nand B	



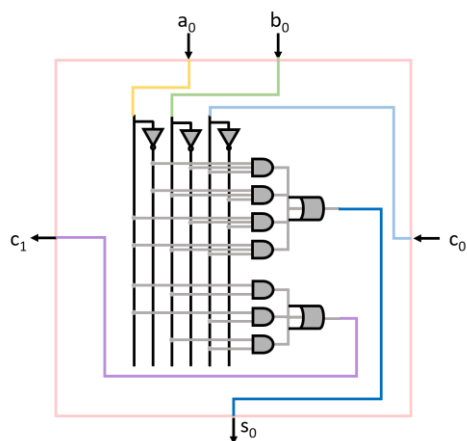
Operations	4 bit (Anegate, Binvert, ALUop)
B sub A	1010
A nand B	1101

例(41)：When you perform addition of two n-bit numbers with 2's complement representation, how do you determine whether there is overflow? Let two words be A and B.

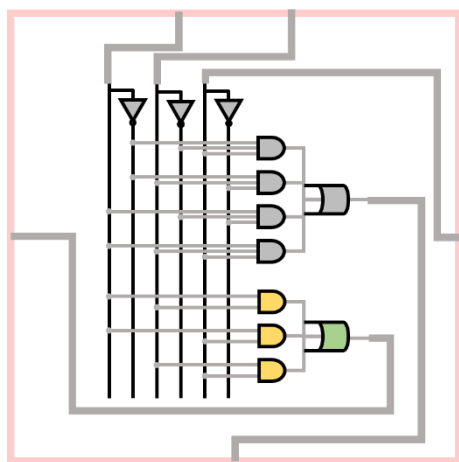
Let the sum be S : $Overflow = a_{n-1}b_{n-1}(-s_{n-1}) + (-a_{n-1})(-b_{n-1})s_{n-1}$

重點五：前瞻進位加法器

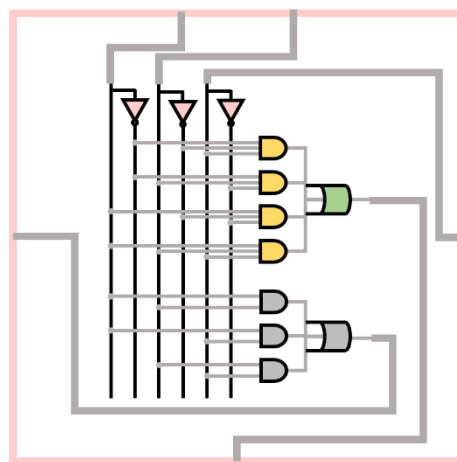
漣波進位加法器



一個 ALU

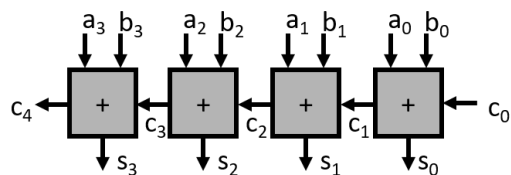
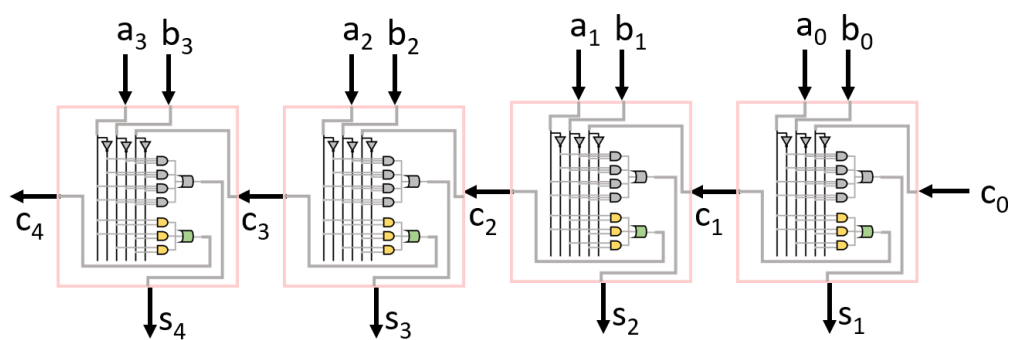


Carry delay : 2 gates (2N gates)



Sum delay : 3 gates (2N+1 gates)

Delay gates 的數量

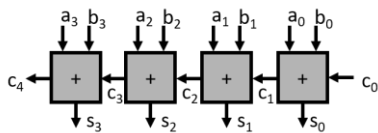


漣波進位加法器

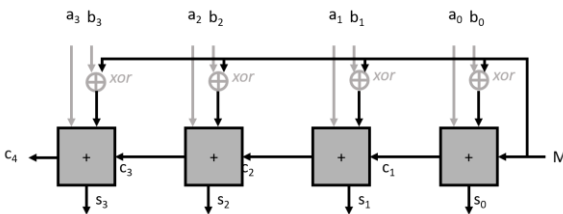
例(54) :

1. Use the block diagram of 1-bit full adder as a basic block to construct a 32-bit ripple adder ($S = A+B$).
2. Add some logic blocks to the design of ripple adder so that it can do 2's complement subtraction ($S = A-B$).
3. Using 4-bit carry-look-ahead blocks to form a 16-bit carry look-ahead adder. Draw the block diagram and write down the corresponding logic equations.
4. Compare the number of "gate delays" for the critical paths of two 16-bit adders, one use ripple carry and one using two-level carry look-ahead.

1. 漣波進位加法器(類似下圖，但延伸至 32 個 ALU)



2. 變型漣波進位加法器 : $S=A+B$ when $M=0$, $S=A-B$ when $M=1$ (因為 $a \text{ xor } 1 = -a$, $a \text{ xor } 0 = a$)

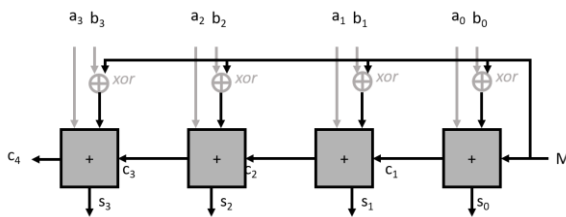


3. CLA

4. $2 \times 16 = 32$

$1+2+5$

例(67) : Draw the schematic for a 4-bit 2's complement adder/subtractor that produces $A+B$ if $K(M)=1$, $A-B$ if $K(M)=0$. In your design try to use minimum number of the following basic logic gates (1-bit adders, AND, OR, INV, and XOR).



因為 $a \text{ xor } 1 = -a$, $a \text{ xor } 0 = a$

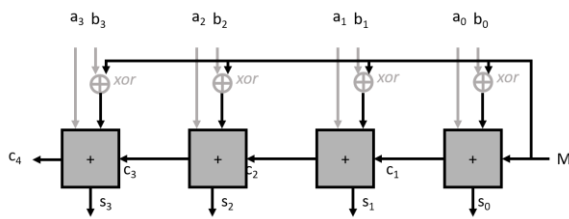
註：因為題目並沒有說可以使用 Mux，所以不能用選擇，需用 INV 與 XOR

例(18) :

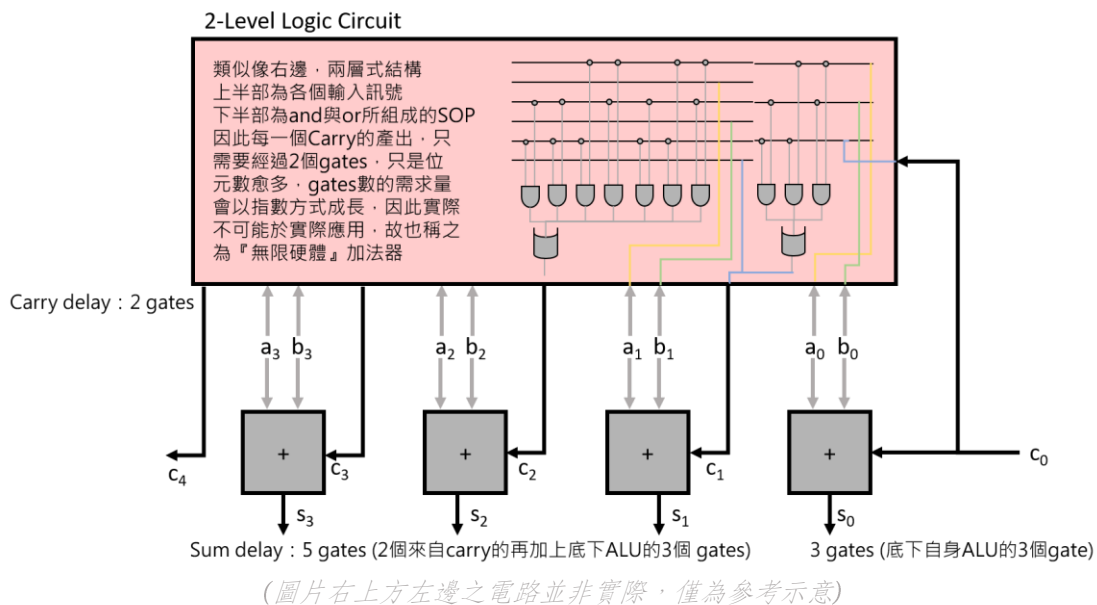
1. Convert the decimal number 0.687 to binary format
2. Sketch a 4-bits Adder-Subtractor Circuit using 4 Full-Adder(FA) and 4 XOR gates. The inputs are (B3B2B1B0), (A3A2A1A0) and the signal S controls the add/sub operation ($S = 0$ for addition)

1. 0.1011

2. 圖中的 M 即為 S



無限(窮)硬體加法器



理想上，我們可以利用代入及推導的方式可以求出 c_n 由最基本 $a_0 \sim a_{n-1}$ 、 $b_0 \sim b_{n-1}$ 與 $c_0 \sim c_{n-1}$ 所組合而成的 SOP，再以硬體拼出，即可透過兩層式結構，快速求得 carry，然而實際上是不可能有無限硬體

$$c_1 = a_0 b_0 + a_0 c_0 + b_0 c_0$$

$$c_2 = a_1 b_1 + a_1 c_1 + b_1 c_1 \quad (c_1 \text{ 代入}) = a_1 b_1 + a_1(a_0 b_0 + a_0 c_0 + b_0 c_0) + b_1(a_0 b_0 + a_0 c_0 + b_0 c_0) \\ = a_1 b_1 + a_1 a_0 b_0 + a_1 a_0 c_0 + a_1 b_0 c_0 + a_0 b_0 b_1 + a_0 b_1 c_0 + b_0 b_1 c_0$$

例(7)：Consider a two-bit adder with its two operands A_i and B_i where $i=0, 1$ (0 is the least significant bit) and the carry-in bit C_0 and carry-out C_2 .

- Derive the Boolean equation for the carry-in C_0 and carry-out bit C_2 using only A_i , B_i (where $i=0, 1$) and C_0
- In a carry-look-ahead adder, there are two additional signals defined, $G_i = A_i \cdot B_i$, $P_i = A_i + B_i$. Derive the Boolean equation for the carry-out bit C_2 using only G_i , P_i (where $i=0, 1$) and C_0 .
- What are the advantages of the Boolean equation in Problem (1) over the Boolean equation in Problem (2)?

1	$c_2 = a_1 b_1 + a_1 a_0 b_0 + a_1 a_0 c_0 + a_1 b_0 c_0 + a_0 b_0 b_1 + a_0 b_1 c_0 + b_0 b_1 c_0$
2	$c_2 = g_1 + p_1 g_1 + p_1 p_0 g_0$
3	The gate delay for 1 and 2 are 2 and 3, respectively. So, the Boolean equation in 1 is faster than in 2.

前瞻進位加法器

有點類似漣波進位加法器與無限硬體加法器的合併

Input			Output	
a	b	Cin	Cout	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

a_i	b_i	c_{i+1}
0	0	0
0	1	c_i
1	0	c_i
1	1	1

kill
propagate
propagate
generate

$$c_{i+1} = a_i b_i + (a_i + b_i) c_i$$

$$c_{i+1} = g_i + p_i c_i$$

$$g_i = a_i b_i$$

$$p_i = a_i + b_i$$

透過右上角的等式代入，便可以得到以下公式

$$c_1 = g_0 + p_0 c_0$$

$$c_2 = g_1 + p_1 g_0 + p_1 p_0 c_0$$

$$c_3 = g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0$$

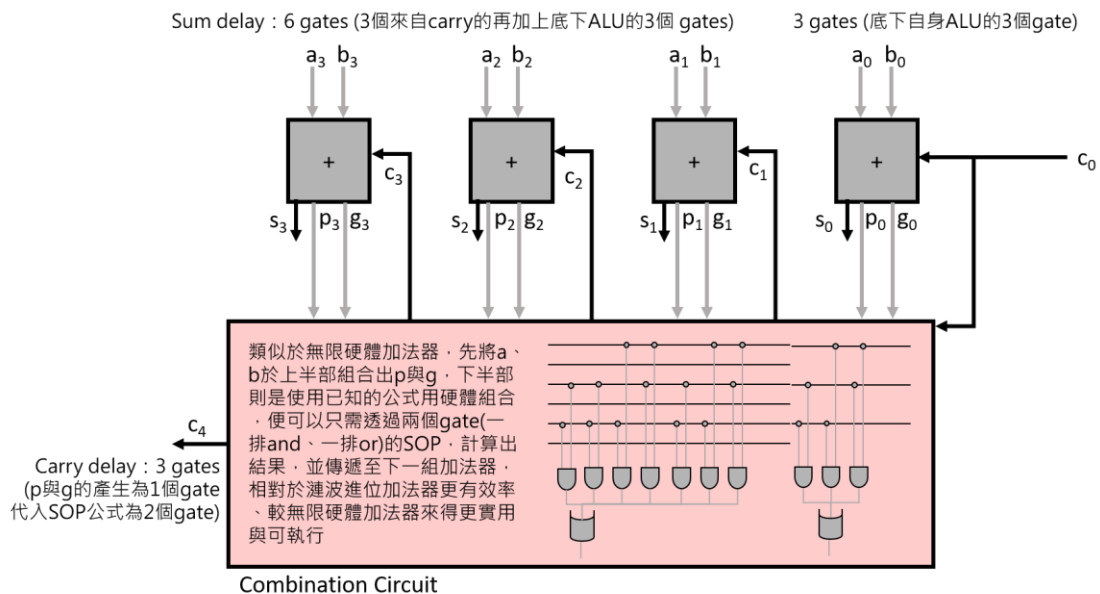
$$c_4 = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 c_0$$

$$c_{i+1} = g_{i+0} + p_{i+0} c_{i+0}$$

$$c_{i+2} = g_{i+1} + p_{i+1} g_{i+0} + p_{i+1} p_{i+0} c_{i+0}$$

$$c_{i+3} = g_{i+2} + p_{i+2} g_{i+1} + p_{i+2} p_{i+1} g_{i+0} + p_{i+2} p_{i+1} p_{i+0} c_{i+0}$$

$$c_{i+4} = g_{i+3} + p_{i+3} g_{i+2} + p_{i+3} p_{i+2} g_{i+1} + p_{i+3} p_{i+2} p_{i+1} g_{i+0} + p_{i+3} p_{i+2} p_{i+1} p_{i+0} c_{i+0}$$



(圖片右上方左邊之電路並非實際，僅為參考示意)

例(37)：Carry load-ahead adder can diminish the carry delay which dominates the delay of ripple carry adder. Generate(g_i) and propagate(p_i) functions are two main operations of carry look-ahead adder. Assume a and b are two operands and c_{i+1} is the carry out of level I and carry in of level I+1, which is (are) correct?

1. $g_i = a_i b_i$
2. $p_i = (a_i + b_i) c_i$
3. if g_i equals to 1, we can say the carry out of level I is 1.
4. Carry look-ahead adder can be extended to multi-level style. The first group generate of a 3-bit group can then be defined as $G_0 = g_2 + (p_2 * g_1) + (p_2 * p_1 * g_0)$

1、3、4

例(45)：Carry look-ahead is often used to speed up the addition operation in ALU. For a 4-bit addition with carry look-ahead, assuming the two 4-bit inputs are $a_3 a_2 a_1 a_0$ and $b_3 b_2 b_1 b_0$, and the carry-in is c_0 .

1. First derive the recursive equations of carry-out c_{i+1} in terms of a_i and b_i and c_i , where $i=0-3$.
2. Then by defining the generate (g_i) and propagate (p_i) signals, express c_1 , c_2 , c_3 , and c_4 in terms of only g_i 's, p_i 's and c_0 .
3. Estimate the speed up for this simple 4-bit carry look-ahead adder over the 4-bit ripple carry adder (assuming each logic gate introduces T delay).

$$1. \quad c_{i+1} = a_i b_i + a_i c_i + b_i c_i$$

$$2. \quad c_1 = g_0 + p_0 c_0$$

$$c_2 = g_1 + p_1 g_0 + p_1 p_0 c_0$$

$$c_3 = g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0$$

$$c_4 = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 c_0$$

3. The critical path delay for 4-bit RCA = $2T * 4 = 8T$. The critical path delay for 4-bit CLA = $2T + T = 3T$. Speedup = $8T/3T = 2.67$

例(63) : Given $A = a_{n-1} \sim a_0$, $B = b_{n-1} \sim b_0$, and carry-in c_0 :

1. What is the equation for g_i , which indicates that a carry c_{i+1} must be generated regardless of c_i ?
2. What is the equation for p_i , which indicates that $c_{i+1} = c_i$?
3. $C_n = f(gx, px, c_0)|_{x=n-1 \sim 0} = ?$

$$1. \quad g_i = a_i b_i$$

$$2. \quad p_i = a_i + b_i$$

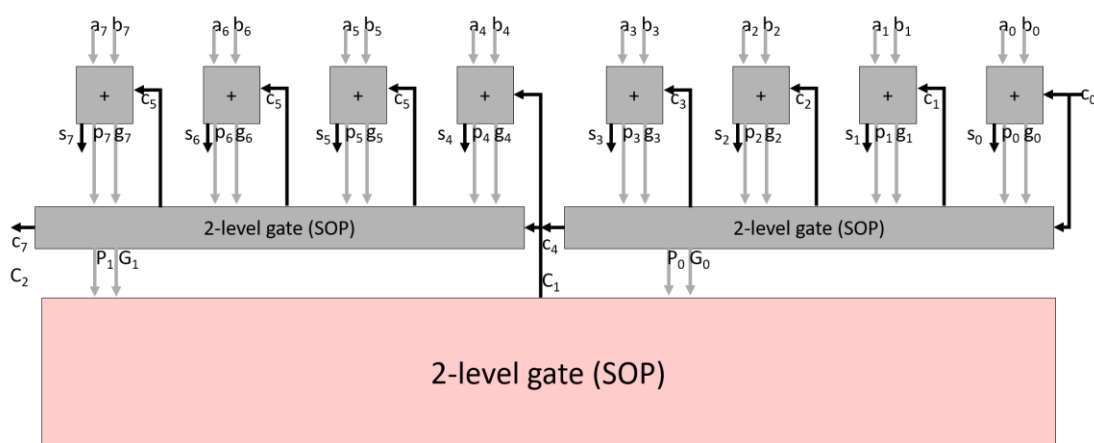
$$3. \quad c_n = g_{n-1} + p_{n-1} g_{n-2} + \dots + p_{n-1} \dots p_0 c_0$$

例(76) : Please design a 4-bit carry look-ahead adder and a ripple carry adder, and compare the delays for these two adders.

略

十六位元前瞻進位加法器

基本上是建立在前瞻進位加法器之上，但因為愈後 bit 會需要過多的 gate(如無限硬體)，故改以雙層，乃至於多層之方式執行



$$P_0 = p_3 p_2 p_1 p_0$$

$$P_1 = p_7 p_6 p_5 p_4$$

$$P_2 = p_{11} p_{10} p_9 p_8$$

$$P_3 = p_{15} p_{14} p_{13} p_{12}$$

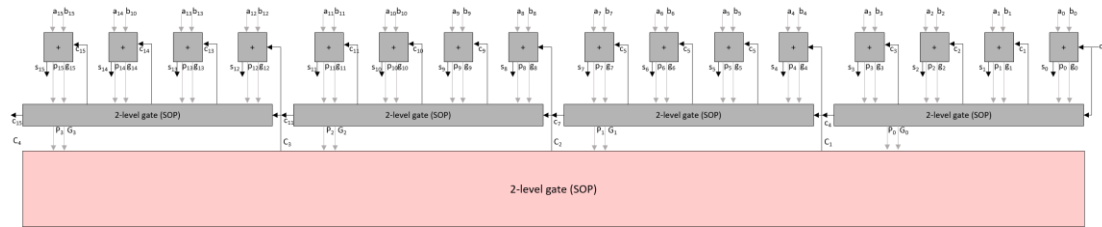
$$G_0 = g_3 + (p_3 g_2) + (p_3 p_2 g_1) + (p_3 p_2 p_1 g_0)$$

$$G_1 = g_7 + (p_7 g_6) + (p_7 p_6 g_5) + (p_7 p_6 p_5 g_4)$$

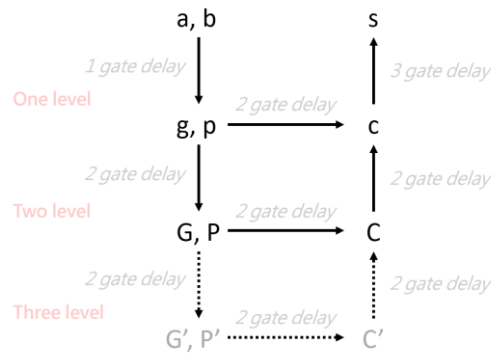
$$G_2 = g_{11} + (p_{11} g_{10}) + (p_{11} p_{10} g_9) + (p_{11} p_{10} p_9 g_8)$$

$$G_3 = g_{15} + (p_{15} g_{14}) + (p_{15} p_{14} g_{13}) + (p_{15} p_{14} p_{13} g_{12})$$

$$\begin{aligned}
C_1(c_{04}) &= G_0 + (c_0 P_0) \\
C_2(c_{08}) &= G_1 + (G_0 P_1) + (c_0 P_0 P_1) \\
C_3(c_{12}) &= G_2 + (G_1 P_2) + (G_0 P_1 P_2) + (c_0 P_0 P_1 P_2) \\
C_4(c_{16}) &= G_3 + (G_2 P_3) + (G_1 P_2 P_3) + (G_0 P_1 P_2 P_3) + (c_0 P_0 P_1 P_2 P_3)
\end{aligned}$$



16bits 加法器完整示意圖



多層次的 Carry 與 Sum 的 gate delay 數目轉換表

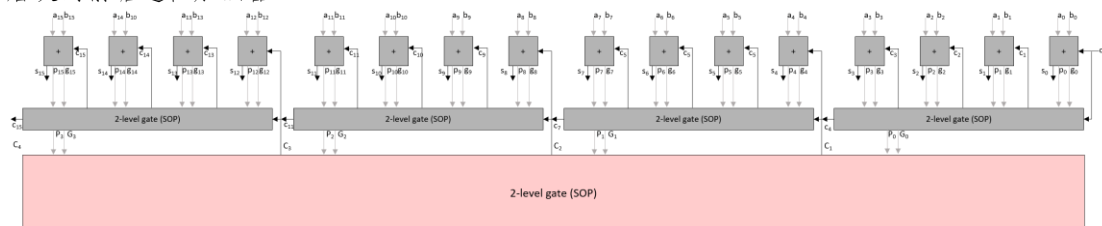
練習：試決定底下兩 16 位元數字的 g_i 、 p_i 、 P_i 和 G_i ，還有 $\text{CarryOut}_{15}(C_4)$ 為何？

a: 0001 1010 0011 0011
b: 1110 0101 1110 1011

a: 0001 1010 0011 0011	P_i : 1110
b: 1110 0101 1110 1011	G_i : 0010
g_i : 0000 0000 0010 0011	C_i : 1xxx
p_i : 1111 1111 1111 1011	x 代表未計算

另解：先切成四塊，若有一組 $a_i + b_i$ 都為零，則該區塊 P 為 0；若該區塊相加有進位則 G 為 1； C 則是看該區塊之前的所有數相加是否進位，若有則 1，若無則 0

練習：試比較兩個 16 位元加法器 gate delay 的數目，其中一個為漣波進位加法器，另一個用兩層次的前瞻進位加法器。



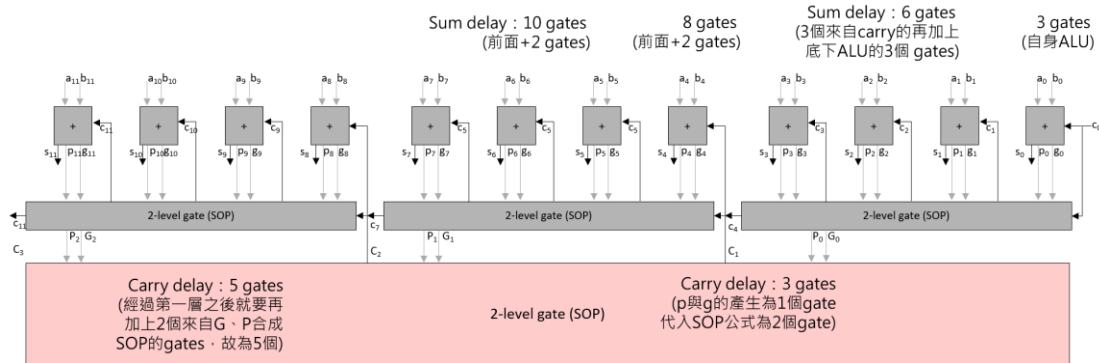
前瞻： $C_4 = 1 + 2 + 2 = 5$ ；漣波： $C_4 = 16 * 2 = 32$

例(31) : Carry Look-ahead Adder (CLA)

Please use 4-bit CLA's to design an efficient 12-bit CLA. Please draw the block diagram first and then describe how signals are generated/ processed step by step to determine the last Carryout (CarryOut11) in the following example:

x: 0001 1010 0011

y: 1110 0101 1110



a:	0001 1010 0011	Pi:	111
b:	1110 0101 1110	Gi:	001
gi:	0000 0000 0010	Ci:	1xx
pi:	1111 1111 1111	x	代表未計算

例(1) : Which of the following statements are true?

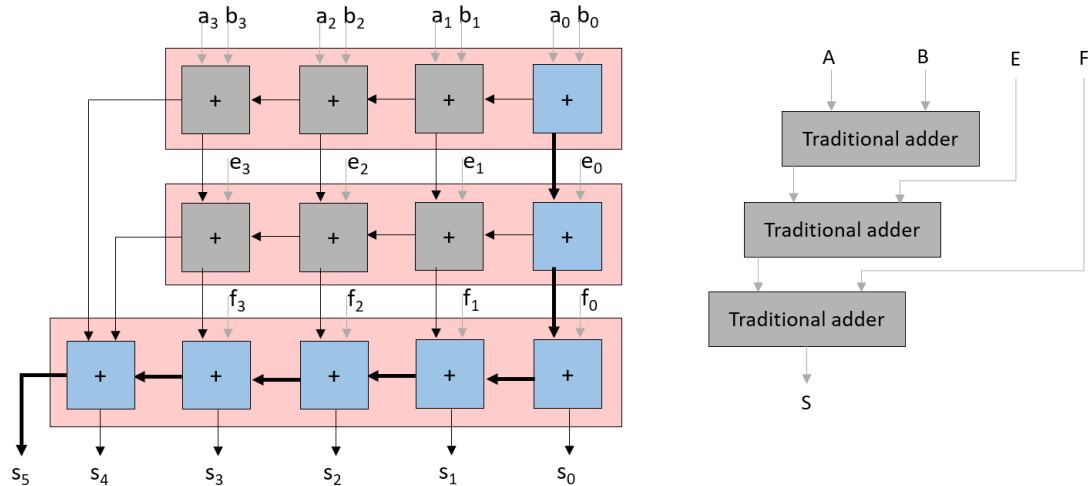
- For a 16-bit addition, if we use ripple adder and assume that the delay from c_i to c_{i+1} of any full adder is 1nsec, then the 16-bit addition takes 16 nsec.
- If we use carry-look-ahead adder, the 16-bit addition is divided into four 4-bit adders. In each 4-bit adder, there is a 4-bit carry-look-head circuit. Assume that there is one gate delay for all g_i and p_i , and two gate delays for each carry c_4 , c_8 , c_{12} , and c_{16} , and finally three gate delays for s_{15} . Then 9 gate delays are required generating c_{16} after X, Y, and c_0 are applied as input.
- Assume that each gate delay requires 0.5 nsec. This 16-bit carry-look-ahead circuit. With a second-level look-ahead circuit, 5 gate delays is required for generating c_{16} after X, Y and c_0 are applied as input.
- Assume that each gate delay requires 0.5 nsec. This 16-bit carry-look-ahead adder requires 2.5 nsec for a single addition operation.

2、4

註 : This 16-bit CLA is formed, like ripple carry adder, by serial connecting the four 4-bit CLA. The delay of c_{16} is $3+2+2+2=9$ gate delay.

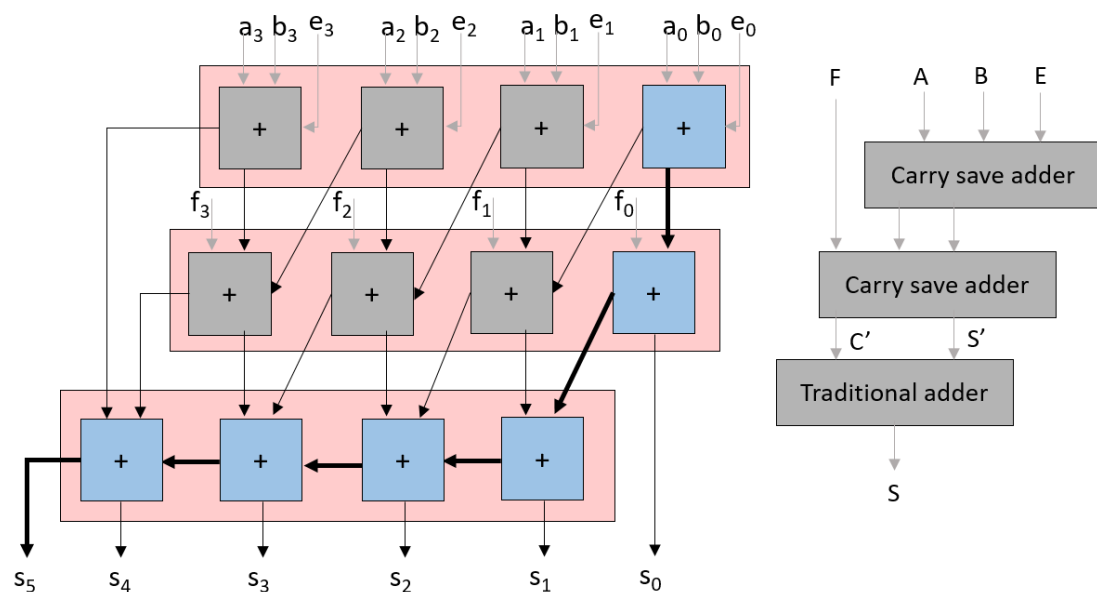
重點六：進位儲存加法器

當我們要將多個數值相加時，直覺上會使用下圖的加法方式：兩兩相加後再兩兩相加，不過在進位部分，由於 **bits** 數相等時，進位的位置也相同，因此可以累積先不處理，等所有原始數值位元加法都完成後，再統一處理進位部分。而以四個數值為例：總時間會需要經過 7 個(藍色的)gates



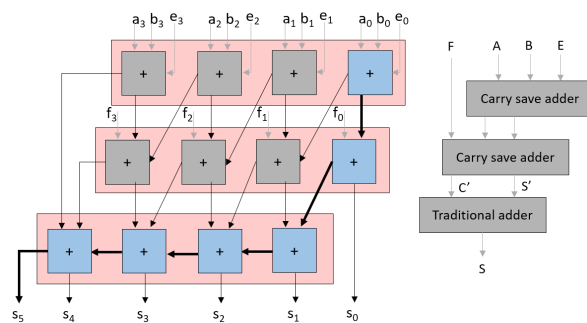
上述方式雖然可完成計算，但成本可以再節省：利用分隔 Carry 與 Sum 的方式，一次進行三個數值的計算。(只需要經過 6 個 gates)

A	1011
B	1100
+ E	1111
S1	1000
C1	1111
+ F	0010
S2	0101
+ C2	1010
	101001



例(68) : We want to add four 4-bit numbers, A[3:0], B[3:0], C[3:0], and D[3:0] together using carry-save addition. Draw the schematic using 1-bit full adders.

把 e 改成 c 、 f 改成 d



例(24) : Let a, b, c be three 8-bit operands to a carry save adder and their values be

$a = 0001\ 0110$, $b = 0110\ 1101$, and $c = 0100\ 1110$

- What are the two outputs of this carry save adder?
- Given the above carry save adder as basic building block, please show the block diagram of 8×8 Wallace-tree multiplier with minimum delay.

1.

a $0001\ 0110$

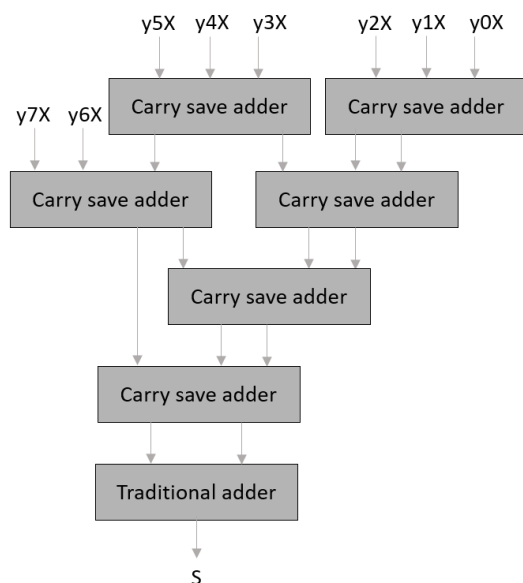
b $0110\ 1101$

c $0100\ 1110$

Sum $0011\ 0101$

Carry $01001\ 110$

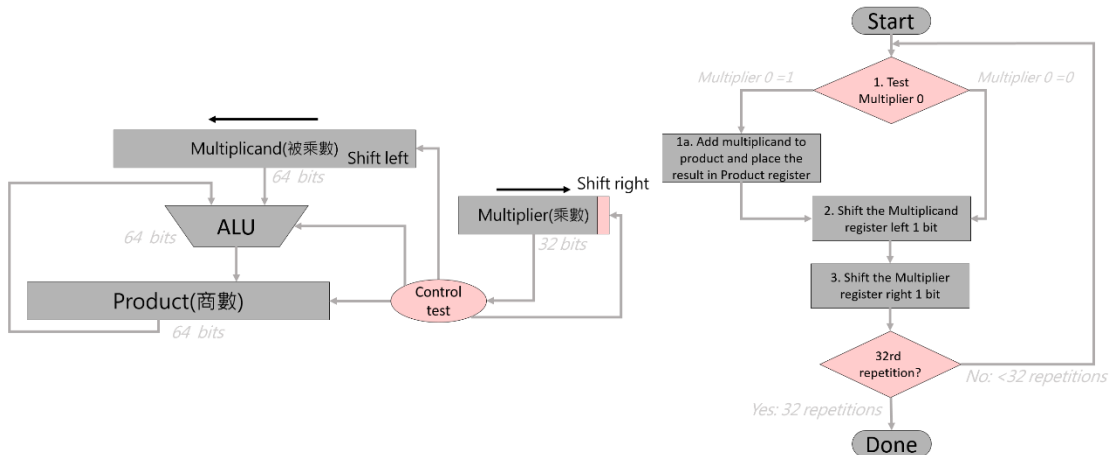
2. Suppose that $X \cdot Y = Z$. We arrange carry save adders to add the 8 partially products as following:



重點七：無號數乘法

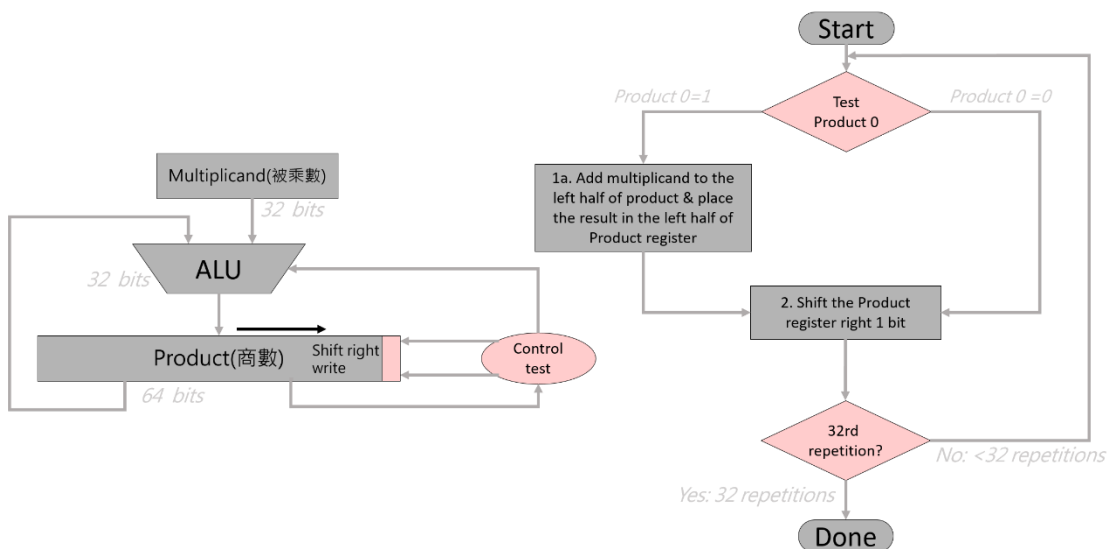
乘法可以是由多次重覆的加法來完成，因此我們在硬體的設計上亦是如此，只不過要特別注意『位數』的部分。無號數的乘法分為 2 種：傳統乘法與硬體最佳化乘法：

傳統乘法：因為要特別注意被乘數的位數，故需要多一倍(64 bits)的位元來儲存，而乘數的執行一次只有 1 bit，所以在執行時需要退位：



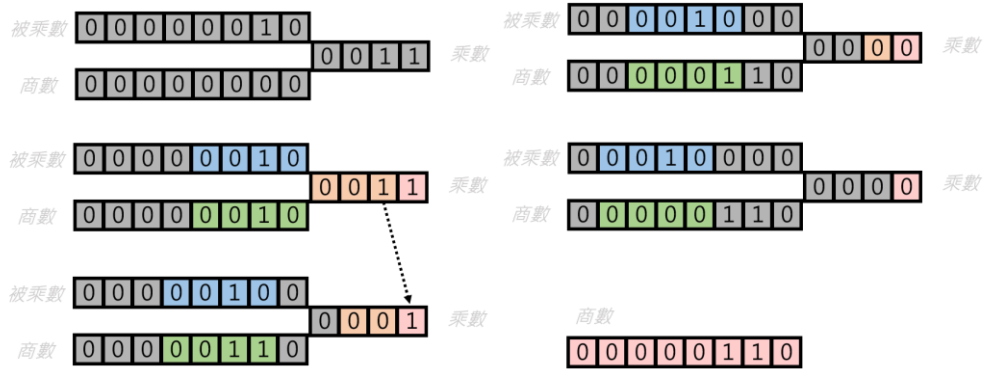
硬體最佳化乘法：

一次要移動被乘數與乘數的效益不高，再加上被乘數需要多一倍的 bits 數，故若是直接將乘數置於 ALU 位置，即可節省上述成本，且一次只需移動商數



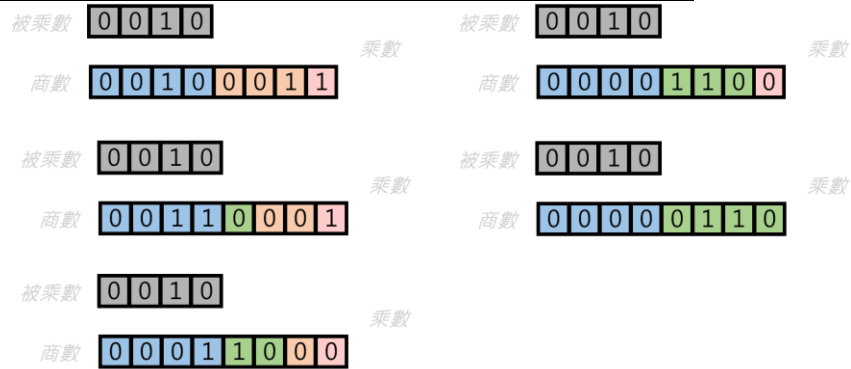
練習：以傳統乘法演算法計算 0010 * 0011，並詳列計算步驟：

Iteration	step	multiplier	multiplicand	product
0	Initial values	0011	0000 0010	0000 0000
1	1a: 1->prod=prod+Mcand	0011	0000 0010	0000 0010
	2:shift left multiplicand	0011	0000 0100	0000 0010
	3:shift right multiplier	0001	0000 0100	0000 0010
2	1a: 1->prod=prod+Mcand	0001	0000 0100	0000 0110
	2:shift left multiplicand	0001	0000 1000	0000 0110
	3:shift right multiplier	0000	0000 1000	0000 0110
3	1a: 1->prod=prod+Mcand	0000	0000 1000	0000 0110
	2:shift left multiplicand	0000	0001 0000	0000 0110
	3:shift right multiplier	0000	0001 0000	0000 0110
4	1a: 1->prod=prod+Mcand	0000	0001 0000	0000 0110
	2:shift left multiplicand	0000	0010 0000	0000 0110
	3:shift right multiplier	0000	0010 0000	0000 0110



練習：以硬體最佳化乘法演算法計算 0010 * 0011，並詳列計算步驟：

Iteration	step	multiplicand	Product
0	Initial values	0010	0000 0011
1	1a: 1->prod=prod+Macand	0010	0010 0011
	2:shift right product	0010	0001 0001
2	1a: 1->prod=prod+Macand	0010	0011 0001
	2:shift right product	0010	0001 1000
3	1a: 1->prod=prod+Macand	0010	0001 1000
	2:shift right product	0010	0000 1100
4	1a: 1->prod=prod+Macand	0010	0000 1100
	2:shift right product	0010	0000 0110

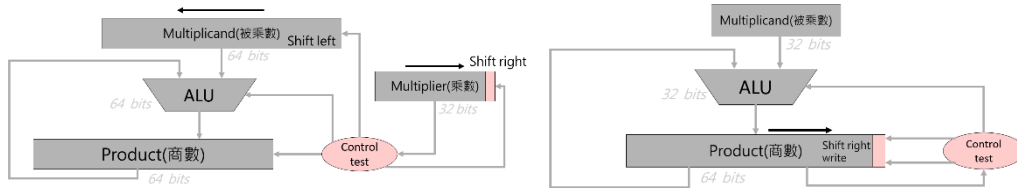


例(6) : Multiply algorithm

1. For a multiplication with multiplier 1100 1011 1110 0111, the traditional multiplication algorithm performs 11 additions for the product.
2. For a multiplication with multiplier 1100 1011 1110 0111, Booth's algorithm performs 3 additions and 3 subtractions for the product
3. Booth's algorithm requires less arithmetic operations than that of the traditional multiplication
4. Booth's algorithm may be applied to two's complement multiplication.

1、4

例(5) : Figure 1 is a basic multiplication hardware and Figure 2 is an improved version of the basic on.



The following statements are possible observations for the improvement:

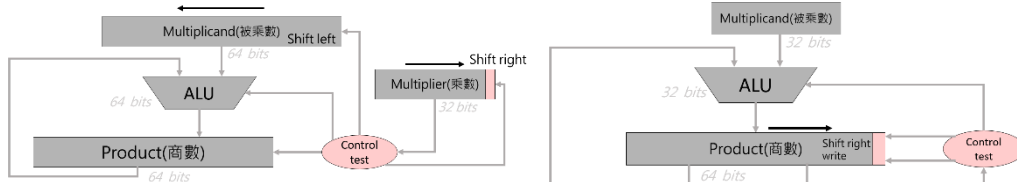
- S1: The multiplicand register, ALU, Multiplier register and the Product register are all 32 bits wide;
- S2: The Product register always shifts right two bits in each iteration;
- S3: The original Multiplier register disappears and is now placed in the right half of the Product register initially;
- S4: The left side of the Product register is initially set to zero.
- S5: The Multiplicand register is initially set to zero.

Please copy the following table to your answer sheet and determine if these statements are true or false.

Statement	T/F	
S1	F	Product register is 64 bits and there is no Multiplier register
S2	F	The Product register always shifts right 1 bits in each iteration
S3	T	
S4	T	
S5	F	The Multiplicand register is initially set to multiplicand

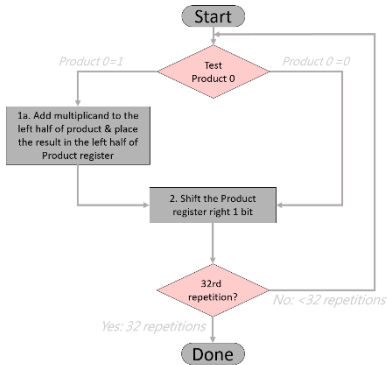
例(55) : The following figures show two multiplication structures. Which is correct?

1. The shift operation in the multiplicand in structure a is shift-right.
2. The shift operation in the multiplier in the structure a is shift-right
3. The multiplier is stored in the right part of the product register in structure b.
4. In structure b, one control signal for shifting multiplicand register is missed.



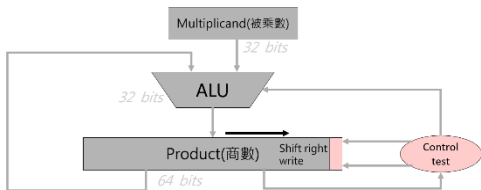
2、註 : the multiplier is stored in the right part of the product register initially but is faded away after multiplication is completion.

例(16)：Draw the flow chart of a 4-bit multiplication algorithm, given that you have a 4-bit ALU, 8-bit product register, and a 4-bit multiplicand register. Write a numerical example to show how your multiplication algorithm works.



Iteration	step	multiplicand	Product
0	Initial values	0010	0000 0011
1	1a:1->prod=prod+Macand	0010	0010 0011
	2:shift right product	0010	0001 0001
2	1a:1->prod=prod+Macand	0010	0011 0001
	2:shift right product	0010	0001 1000
3	1a:1->prod=prod+Macand	0010	0001 1000
	2:shift right product	0010	0000 1100
4	1a:1->prod=prod+Macand	0010	0000 1100
	2:shift right product	0010	0000 0110

例(17)：Draw the block diagram of a 32-bit multiplier, and explain how it works. Function units you can use are registers, adders, and other control logics.

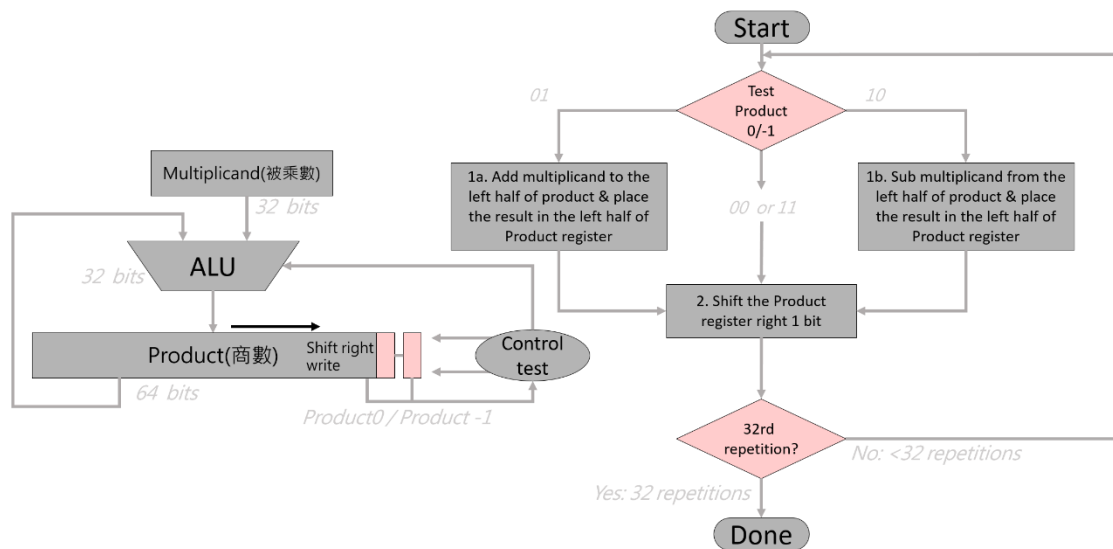


1. Place multiplicand in the Multiplicand register and multiplier in the right half of the Product register.
2. Test bit 0 of the Product register; if 0 then do nothing else add multiplicand to the left half of the product and place the result in the left half of the Product register.
3. Shift the Product register right 1 bit.
4. Go to 2. if repetitions have not done.

重點八：有號數乘法

布斯演算法：建立在硬體最佳化演算法之硬體上，利用多加的一 Product -1 bit，來進行加或減的有號數乘法：

- 00：不執行運算
- 01：將被乘數加到乘積左半部
- 10：將被乘數減到乘積左半部
- 11：不執行運算



證明：

ai	ai+1	Operation	ai+1 - ai	Operation
0	0	Do nothing	0	Do nothing
0	1	add b	+1	add b
1	0	sub b	-1	sub b
1	1	Do nothing	0	Do nothing

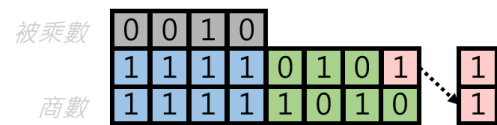
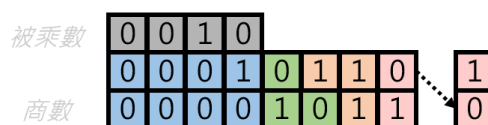
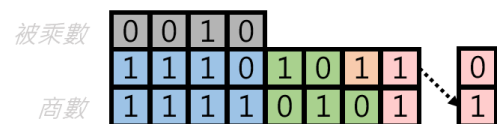
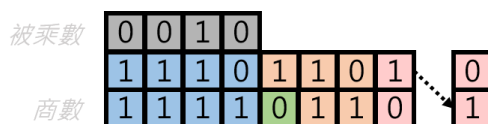
32-bits 布斯演算法可寫成： $= \sum_{i=0}^{31} (a_{i+1} - a_i) \times b \times 2^i$

全部展開後 = $b \times [a_{31} \times -2^{31} + (a_{i+1} \times 2^{30} + \dots + a_0 \times 2^0)]$

即是有號數的 2 補數表示法

練習：利用布斯演算法執行 $2 \times -3 = -6$ 或 $0010 \times 1101 = 1111\ 1010$

Iteration	step	multiplicand	Product
0	Initial values	0010	0000 1101 0
1	1c: 10-> prod=prod-Mcand	0010	1110 1101 0
	2: shift right product	0010	1111 0110 1
2	1b: 10-> prod=prod+Mcand	0010	0001 0110 1
	2: shift right product	0010	0000 1011 0
3	1c: 10-> prod=prod-Mcand	0010	1110 1011 0
	2: shift right product	0010	1111 0101 1
4	1d: 11-> no operation	0010	1111 0101 1
	2: shift right product	0010	1111 1010 1

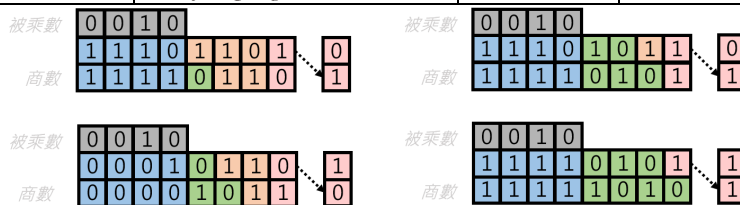


例(36) : The following questions are about computer arithmetic.

1. Perform $-2 * 3$ (both numbers are given in decimal) by first converting the numbers to the 4-bit 2's complement ones and then using Booth's algorithm to get the 8-bit product.
2. Convert the IEEE 754 single precision floating-point number C0F4 0000, represented in hexadecimal, to its equivalent decimal number.

1. $-2 * 3 = 1110 * 0011 = 1111\ 1010 = -6$

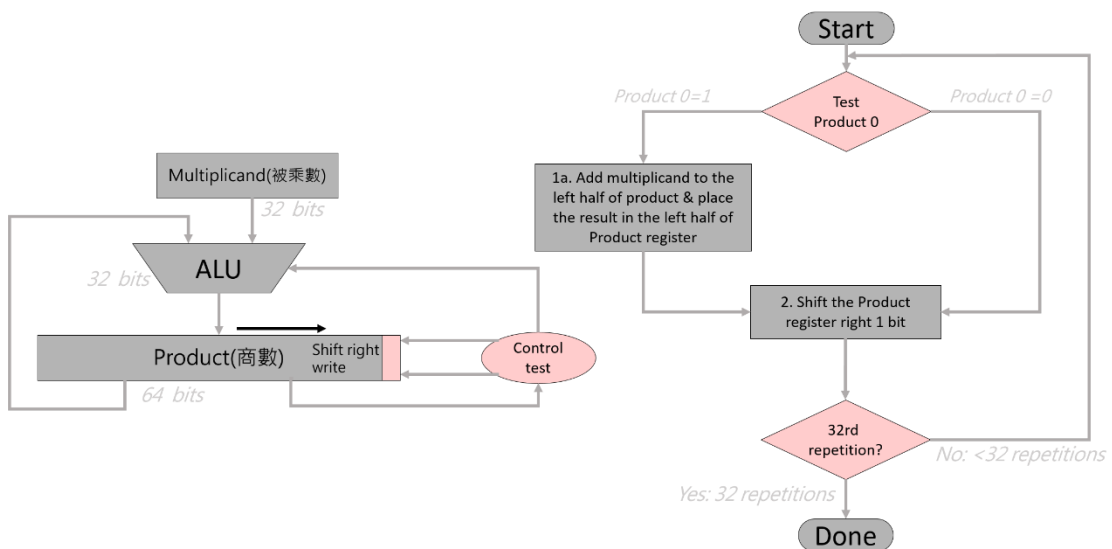
2. Iteration	step	multiplicand	Product
0	Initial values	0010	0000 1101 0
1	1c:10-> prod=prod-Mcand	0010	1110 1101 0
	2:shift right product	0010	1111 0110 1
2	1b:10-> prod=prod+Mcand	0010	0001 0110 1
	2:shift right product	0010	0000 1011 0
3	1c:10-> prod=prod-Mcand	0010	1110 1011 0
	2:shift right product	0010	1111 0101 1
4	1d:11-> no operation	0010	1111 0101 1
	2:shift right product	0010	1111 1010 1



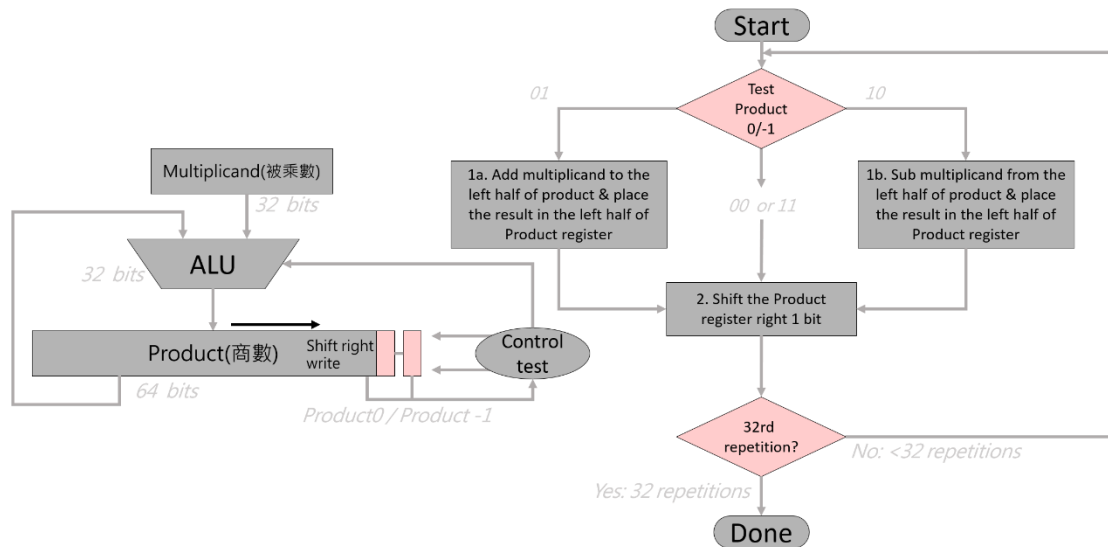
3. C0F4 0000 = 1100 0000 1111 0100 0000 0000 0000 0000
 $= -1 * 1.11101 * 2^{129-127}$

例(64) : Given the hardware and the flow chart of a multiplication algorithm,

1. Modify the hardware as less as possible for Booth's multiplication algorithm. Draw the modified hardware.
2. Based on the modified hardware designed in 1., redraw the flow chart for Booth's multiplication algorithm.
3. Describe the characteristics and advantages of Booth's algorithm.



1. 圖
2. 圖



3. The major features of Booth's algorithm are that it handles signed number multiplication and if shifting is faster than addition, it would be faster than traditional sequential multiplier in average case.

例(53) : (簡化)Is Booth's algorithm always better? Why does the Booth's algorithm work for multiplication of 2's complement signed integers?

1. No, 當 0、1 交錯的時候就不會優於傳統乘法演算法
2. 證明 :

32-bits 布斯演算法可寫成
$$:= \sum_{i=0}^{31} (a_{i-1} - a_i) \times b \times 2^i$$

 全部展開後 = $b \times [a_{31} \times -2^{31} + (a_{i-1} \times 2^{30} + \dots + a_0 \times 2^0)]$
 即是有號數的 2 補數表示法

例(35) :

1. What is the major features of Booth's algorithm?
 2. Please list the worst case of Booth's algorithm for A*B, where A and B are 16-bit data.
 3. Please list the best case of Booth's algorithm for A*B, where A and B are 16-bit data.
1. The major features of Booth's algorithm are that it handles signed number multiplication and if shifting is faster than addition, it would be faster than traditional sequential multiplier in average case.
 2. When B=0101 0101 0101 0101, the Booth's algorithm encounters a worst case since total 16 times of addition/subtraction is needed for this multiplication.
 3. When B= 1111 1111 1111 1111 (or 1000 0000 0000 0000) the Booth's algorithm encounters a best case since only 1 sub is needed for this multiplication. Of course a string of 0s already avoids arithmetic, so we can leave these alone.

例(59) :

1. Fill in the following table using the index provided in the keywords 1-5 to determine the 3-bit Booth algorithm. Assume that you have both the multiplicand and twice the multiplicand already in registers.
 (1.None、2.Add the multiplicand、3.Add twice the multiplicand、4.Sub the multiplicand、5.Sub twice the multiplicand)

- Assume x is 010101 and y is 011011. Please use 2-bit and 3-bit Booth algorithm to do the $y \times x$ operation.
- Will the 3-bit Booth algorithm always have fewer operations than the 2-bit Booth algorithm? Justify your answer by a brief description.

1.

Current bits		Previous bit	Operation
a_{i+1}	a_i	a_{i-1}	
0	0	0	1
0	0	1	2
0	1	0	2
0	1	1	3
1	0	0	5
1	0	1	4
1	1	0	4
1	1	1	1

2.2-bit

Iteration	Step	Multiplicand	Product
0	Initial values	011011	000000 010101 0
1	10->-Multiplicand	011011	100101 010101 0
	Shift right product	011011	110010 101010 1
2	01->+Multiplicand	011011	001101 101010 1
	Shift right product	011011	000110 110101 0
3	10->-Multiplicand	011011	101011 110101 0
	Shift right product	011011	110101 111010 1
4	01->+Multiplicand	011011	010000 111010 1
	Shift right product	011011	001000 011101 0
5	10->-Multiplicand	011011	101101 011101 0
	Shift right product	011011	110110 101110 1
6	01->+Multiplicand	011011	010001 101110 1
	Shift right product	011011	001000 110111 0

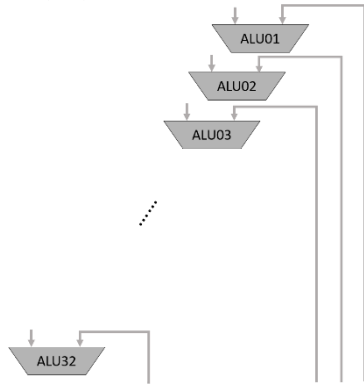
0 1 1 0 1 1	0 0 0 0 0 0	0 1 0 1 0 1	0
1 0 0 1 0 1	0 1 0 1 0 1	0 1	0
1 1 0 0 1 0	1 0 1 0 1 0	1	1
0 1 1 0 1 1	1 1 0 0 1 0	0 1 1 1 0 1 0	1
1 1 0 0 1 0	0 1 1 1 0 1	0 1 1 1 0 1	1
0 0 1 1 0 1	1 0 1 0 1 0	1	0
0 0 0 1 1 0	1 1 0 1 0 1	0	1
0 1 1 0 1 1	0 0 1 0 0 0	0 1 1 1 0 1	0
1 1 0 1 1 0	1 0 1 1 0 1	0	0
1 1 0 1 1 0	1 0 1 1 0 1	1	1
0 1 1 0 1 1	0 0 1 0 0 0	0 1 1 1 0 1	1
1 1 0 1 1 0	1 0 1 1 0 1	1	0
1 1 0 1 1 0	1 0 1 1 0 1	1	1
0 1 1 0 1 1	0 0 1 0 0 0	0 1 1 1 0 1	0
1 1 0 1 1 0	1 0 1 1 0 1	1	1
1 1 0 1 1 0	1 0 1 1 0 1	1	0

3-bits :

Iteration	Step	Multiplicand	Product
0	Initial values	011011	000000 010101 0
1	010->-Multiplicand	011011	011011 010101 0
	Shift right product	011011	000110 110101 0
2	010->+Multiplicand	011011	100001 110101 0
	Shift right product	011011	001000 011101 0
3	010->+Multiplicand	011011	100011 011101 0
	Shift right product	011011	001000 110111 0

左為一般的 Fast Multiplier、右為平行 Fast Multiplier，只需更少的 gates 時間(依是否進位決定)

例(34) : What is the functionality of the following design? What is the benefit of the design?



1. This design is a multiplier array of adders (Fast multiplier)
2. Faster than sequential multiplier
Easy to optimizations to gain further improvements
Easy to pipeline

練習 : In the following table, A represents the bit width of an integer, and B represents the number of time units (tu) taken to perform a step of an operation.

	A (bit width)	B (time units)
a.	4	3 tu
b.	32	7 tu

1. Calculate the time necessary to perform a multiply using the traditional multiplication approach if an integer is A bits wide and each step of the operation takes B time units. Assume that in step 1a an addition is always performed-either the multiplicand will be added, or a 0 will be. Also assume that the registers have already been initialized (you are just counting how long it takes to do the multiplication loop itself). If this is being done in hardware, the shifts of multiplicand and multiplier can be done simultaneously. If this is being done in software, they will have to be done one after the other. Solve for each case.
2. Calculate the time necessary to perform a multiply using the Fast multiplication approach (31 adders stacked vertically) if an integer is A bits wide and an adder takes B time units.
3. Calculate the time necessary to perform a multiply using the Parallel Fast multiplication, if an integer is A bits wide and an adder takes B time units.

1. It takes 1 cycle to do the add, 1 cycle to do the shift, and 1 cycle to decide if we are done. The loop takes $(3 \cdot A)$ cycles.
Software: it takes 1 cycle to do the add, 1 cycle to do each shift, and 1 cycle to decide if we are done. The loop takes $(4 \cdot A)$ cycles.

a.	Hardware: $(3 \cdot 4) \cdot 3tu = 36$ time units Software: $(4 \cdot 4) \cdot 3tu = 48$ time units
b.	Hardware: $(3 \cdot 32) \cdot 7tu = 672$ time units Software: $(4 \cdot 32) \cdot 7tu = 896$ time units

2. There will be $A-1$ adders.

a.	$(4-1) \cdot 3tu = 3 \cdot 3tu = 9$ time units.
b.	$(32-1) \cdot 7tu = 31 \cdot 7tu = 217$ time units.

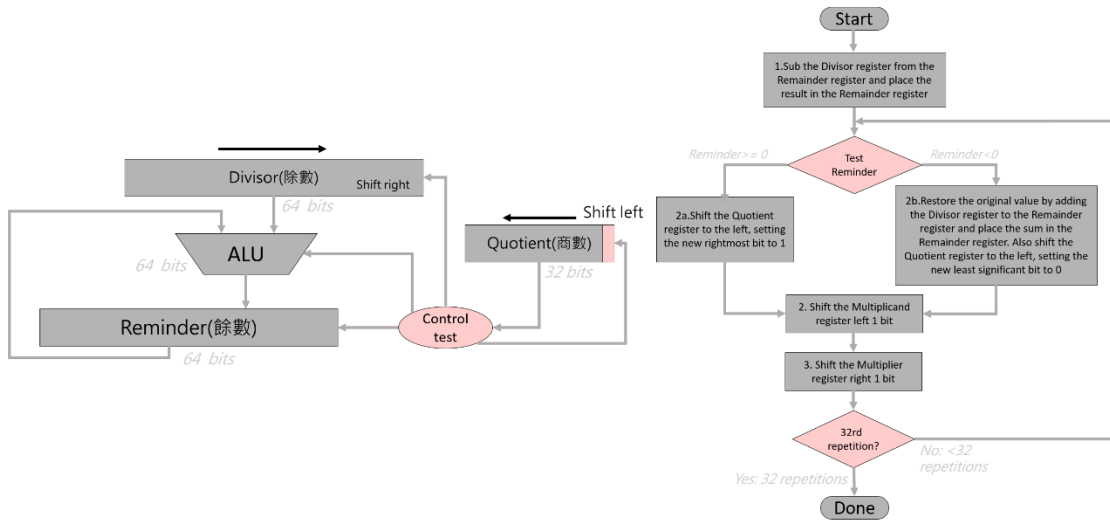
3. The adders are arranged in a tree structure and in $\log_2(A)$ levels.

a.	$\log_2(4) \cdot 3tu = 2 \cdot 3tu = 6$ time units.
b.	$\log_2(32) \cdot 7tu = 5 \cdot 7tu = 35$ time units.

重點九：除法

除法可拆解成連續位元減法，若減超過則加回來再退位。與乘法相同，分成：傳統除法、硬體最佳化除法

傳統除法：



練習：使用傳統除法演算法計算 7 除以 2，或 0111 除以 0010

Iteration	step	quotient	divisor	Remainder
0	Initial values	0000	0010 0000	0000 0111
1	1a: rem=rem-div	0000	0010 0000	1110 0111
	2b: rem<0 -> +div, sll Q, Q0=0	0000	0010 0000	0000 0111
	3: shift div right	0000	0001 0000	0000 0111
2	1a: rem=rem-div	0000	0001 0000	1111 0111
	2b: rem<0 -> +div, sll Q, Q0=0	0000	0001 0000	0000 0111
	3: shift div right	0000	0000 1000	0000 0111
3	1a: rem=rem-div	0000	0000 1000	1111 1111
	2b: rem<0 -> +div, sll Q, Q0=0	0000	0000 1000	0000 0111
	3: shift div right	0000	0000 0100	0000 0111
4	1a: rem=rem-div	0000	0000 0100	0000 0011
	2b: rem≥0 -> -div, sll Q, Q0=1	0001	0000 0100	0000 0011
	3: shift div right	0001	0000 0010	0000 0011
5	1a: rem=rem-div	0011	0000 0010	0000 0001
	2b: rem≥0 -> -div, sll Q, Q0=1	0011	0000 0010	0000 0001
	3: shift div right	0011	0000 0001	0000 0001

0010 0000
1110 0111
0000 0111

0000 0111
0000 0111
0000 0111

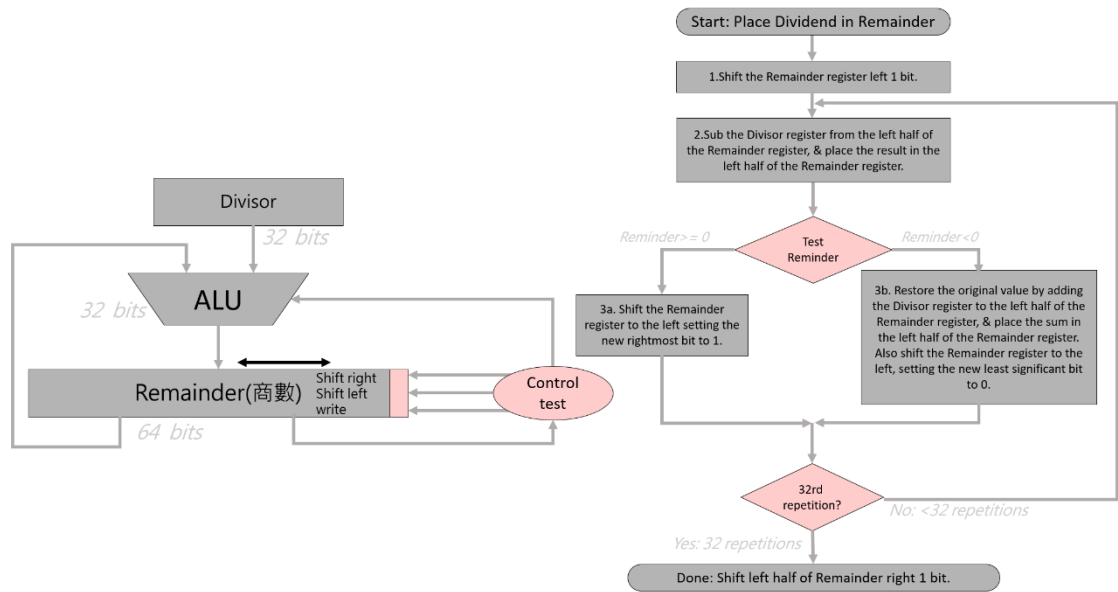
0001 0000
1110 0111
0000 0111

0000 0111
0000 0111
0000 0111

0000 0011
1110 0111
0000 0111

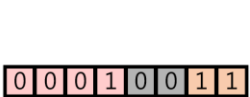
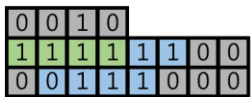
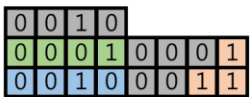
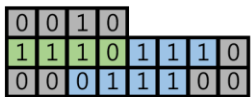
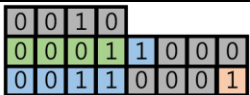
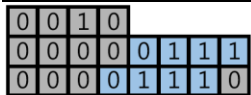
0000 0011
0000 0011
0000 0011

硬體最佳化除法

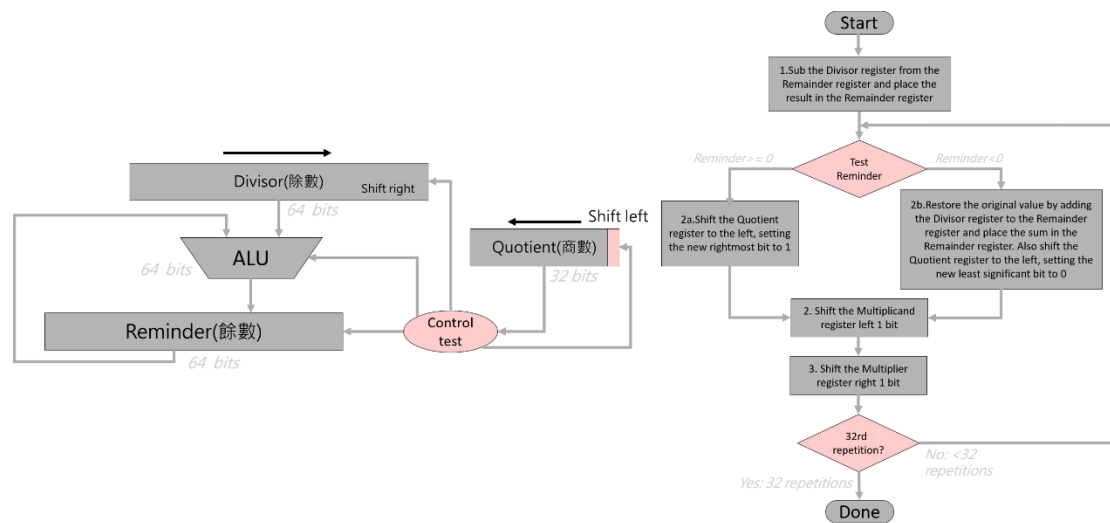


練習：使用硬體最佳化除法演算法計算 7 除法 2，或 0111 除以 0010

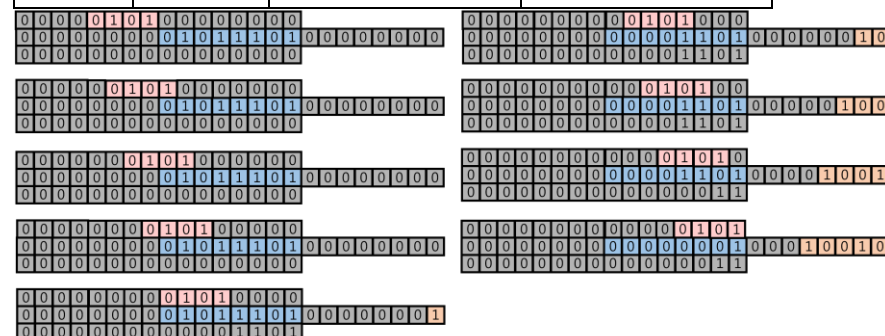
Iteration	step	divisor	Remainder
0	Initial values	0010	0000 0111
	Shift rem left 1	0010	0000 1110
1	2: rem=rem-div	0010	1110 1110
	3b: rem<0 -> +div, sll R, R0=0	0010	0001 1100
2	2: rem=rem-div	0010	1111 1100
	3b: rem<0 -> +div, sll R, R0=0	0010	0011 1000
3	2: rem=rem-div	0010	0001 1000
	3b: rem≥0 -> sll R, R0=0	0010	0011 0001
4	2: rem=rem-div	0010	0001 0001
	3b: rem≥0 -> sll R, R0=0	0010	0010 0011
	Shift left half of rem right 1	0010	0001 0011



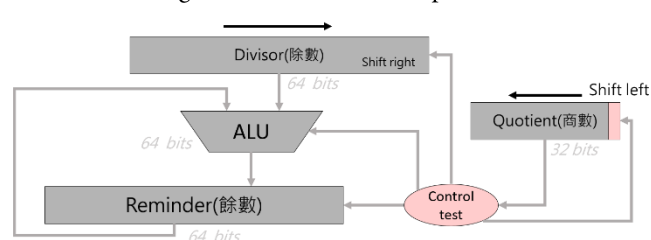
例(20) : Refer to below figure, show each step and final value of each register in calculating 0101 1101 / 0101



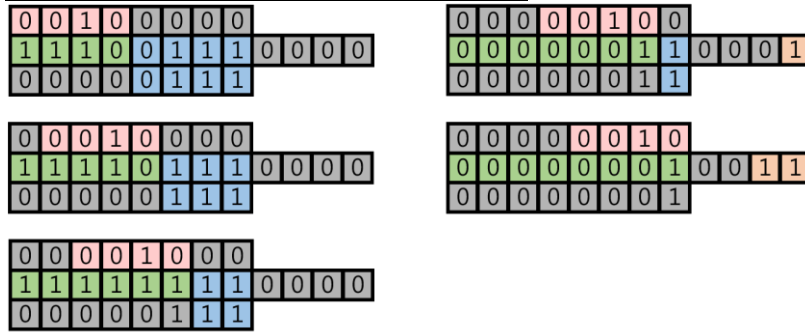
Iteration	Quotient	Divisor	Remainder
0	0000 0000	0000 0101 0000 0000	0000 0000 0101 1101
1	0000 0000	0000 0010 1000 0000	0000 0000 0101 1101
2	0000 0000	0000 0001 0100 0000	0000 0000 0101 1101
3	0000 0000	0000 0000 1010 0000	0000 0000 0101 1101
4	0000 0000	0000 0000 0101 0000	0000 0000 0101 1101
5	0000 0001	0000 0000 0010 1000	0000 0000 0000 1101
6	0000 0010	0000 0000 0001 0100	0000 0000 0000 1101
7	0000 0100	0000 0000 0000 1010	0000 0000 0000 1101
8	0000 1001	0000 0000 0000 0101	0000 0000 0000 0011
9	0001 0010	0000 0000 0000 0010	0000 0000 0000 0011



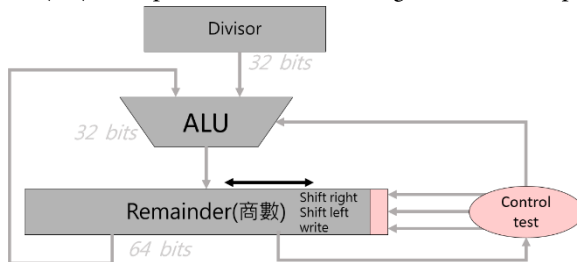
例(38) : Given the 4-bit restoring division block diagram below. Please divide 7 by 2 and show the value of each register for each of the steps.



Iteration	Quotient	Divisor	Remainder
0	0000	0010 0000	0000 0111
1	0000	0001 0000	0000 0111
2	0000	0000 1000	0000 0111
3	000	0000 0100	0000 0111
4	0001	0000 0010	0000 0011
5	0011	0000 0001	0000 0001



例(42) : Explain how the following hardware can perform both 32-bit multiplications and divisions.



Multiplication can be performed on the hardware as follows.

1. Place multiplicand in the 32-bit register and multiplier in the right half of the 64-bit register.
2. Test bit 0 of the 64-bit register; if 0 then do nothing else add multiplicand to the left half of the product and place the result in the left half of the 64-bit register.
3. Shift the 64-bit register right 1 bit.
4. Go to 2. If 32 repetitions have not done.

Division can be performed on the hardware as follows.

1. Place divisor in the 32-bits register and dividend in the right half of the 64-bits register.
2. Shift the 64-bit register left 1 bit.
3. Sub the 32-bit register from the left half of the 64-bit register.
4. If the value in 64-bit register ≥ 0 then shift the 64-bit register to left, setting the new rightmost bit to 1 else restore the original value of the 64-bit register and shift the 64-bit register to left, setting the new rightmost bit to 0.
5. Go to 3. If 32 repetitions have not done.
6. Shift left half of 64-bit register right 1 bit.

MIPS 的乘除法

MIPS 提供 3 個特殊暫存器，除了 PC 之外，便是 Hi、Lo：乘法時合放 Hi、Lo，除法時餘數放 Hi、商數放 Lo；但因為是乘除結果可能為 64-bits，故需要特殊指令(不過其實通常不會用到，故還是以 *mul \$s0, \$s1, \$s2* 為主)

mult \$s2, \$s3 //結果合放 //Hi、Lo	div \$s2, \$s3 //Lo <- \$s2 / \$s3 //Hi <- \$s2 % \$s3
mfhi \$s0 mflo \$s1	

有號數除法：因為過於複雜，故先都以無號數計算完後再轉成有號數，不過『餘數的符號要與被除數相同』

Ex : $-7/2 = -3 \dots -1(O)$

$-7/2 = -4 \dots +1(X)$

重點十：非回復型除法

十年沒考，略過~

重點十一：浮點數的表示

IEEE 754 規範了兩種表示法：1 個 word 儲存的單精度表示法、與 2 個 words 儲存的倍精度表示法。兩者的 MSB 皆表示正負號(0 為正、1 為負)；單精度與倍精度分別以 8/11 個 bits 表示指數數值($2^7 \sim -2^7+1$ 與 $2^{10} \sim -2^{10}+1$)；最後 23/52 bits 表示有效數字，其公式如下：

$$(-1)^{\text{Sign}} * (1 + \text{Fraction}) * 2^{(\text{Exponent} - \text{Bias})}$$

例(10)：A designer of a floating-point represent must find a compromise between the size of the fraction and the size of the exponent. The tradeoff is between precision and range. What does that mean?

Increasing the size of the fraction enhances the precision of the fraction, while increasing the size of the exponent increase the range of numbers that can be represented.

例(30)：Determine if each of the following statements is true(T) or false(F).

1. In the IEEE 754 floating-point representation, the precision of represented numbers is determined by the size of fraction.
2. In computer arithmetic, overflow occurs when adding two positive numbers and the sum is negative.

True、True

例(12)：Which of the following statements about the IEEE single-precision floating point computer numbering format are not correct?

- a. There are 1 bit in sign, 8 bits in exponent, and 24 bits in mantissa.
- b. If sign = 0, exponent = 0111 1101, and significand = 0100 0000 0000 0000 0000 000, then the number represented is 0.3125
- c. The largest positive number that can be represented is $2(1 - 2^{-24}) * 2^{127}$
- d. The smallest positive number that can be represented is $1 * 2^{-126}$
- e. All of the above are correct

a 、 d 、 e

練習：以 IEEE 754 單精度表示法表示 -0.75

1 0111 1110 1000 0000 0000 0000 0000 000

例(44)：IEEE 754 binary representation of a 32-bit floating number is shown below.

1、8、23 bits

What is the correct binary presentation of (-0.75) in IEEE single-precision floating point format?

1 0111 1110 1000 0000 0000 0000 0000 000

例(61)：What is the biased single precision IEEE 754 floating point format of 0.9375? What is purpose to bias the exponent of the floating point numbers?

1. $0.9375 = 0.1111_2 = 1.111 \times 2^{-1}$
2. 多數浮點數運算需要先比較指數大小來決定要對齊哪一個數的指數，使用 bias 表示法可以直接比較指數欄位的無號數值，即可判斷兩個浮點數指數的大小而不需考慮其正負號，因此可以加快比較速度

例(58)：

1. Show the IEEE 754 binary representation for the floating-point number 0.1 in single precision.
2. Add 2.56×10^2 to 2.34×10^4 , assuming that we have only 3 significant decimal digits (No guard and round digits are used)
3. What's the number of ulp (units in the last place) in 2.
4. Assume that you have only 4 significant decimal digits. Round the number 12.4650 to nearest even.

1. $0.1 = 0.0001_2 = 1.0001 \times 2^{-4}$
 $Sign=0, Fraction=.10001, Exponent=-4+127=123$
 $\Rightarrow 0\ 0111\ 1011\ 1001\ 1001\ 1001\ 1001\ 100$
2. $2.56 \times 10^2 + 2.34 \times 10^4 = 0.02 \times 10^4 + 2.34 \times 10^4 = 2.36 \times 10^4$
3. 2
4. 12.46

例(75)：Why is biased notation used in IEEE 754 representation?

多數浮點數運算需要先比較指數大小來決定要對齊哪一個數的指數，使用 bias 表示法可以直接比較指數欄位的無號數值，即可判斷兩個浮點數指數的大小，而不需考慮其正負號，因此可以加速比較速度

練習：以下單精度的浮點數所代表的十進位數為何？

1 1000 0001 0100 0000 0000 0000 0000 000

-5.0

例(74)：

1. Compute the value of the following floating-point number A based on the IEEE standard. (Note: this floating-point number is composed of three fields, i.e., a sign bit, 8 exponent bits, and 23

significant bits)

- Consider the addition process of the following two binary numbers A and B. Determine the so-called carry generate and carry propagate, signals for each bit.

A = (0001 1010)

B = (1110 0101)

- Consider the process of adding up two floating-point numbers in a microprocessor

甲、Derive the proper sequence of the following three operations.

i. Addition of significands

ii. Normalization

iii. Alignment of Exponents

乙、What operation is still needed in addition to the above three operations.

- $(-1) \times (1.01) \times 2^{129-127} = -101 = -5$

- 如表

A = 0001 1010
B = 1110 0101
p = 1111 1111
g = 0000 0000

- a: 312 b: Round the sum

浮點數的編碼：

Single precision		Double precision		Object represented
Exponent	Fraction	Exponent	Fraction	
0	0	0	0	+0
0	nonzero	0	nonzero	+De-normalized number
1-254	anything	1-2046	anything	+Floating-point number
255	0	2047	0	+Infinity
255	nonzero	2047	nonzero	NaN (Not a Number)

大於正最大、小於負最小稱為 Overflow；介於正最小、負最大，稱為 Underflow(非正規化數為 Underflow)

例(57)：Define zero, de-normalized number, floating point number, infinity, and NaN (Not a number) in IEEE 754 double precision format by giving the range of their exponents and significands, respectively. Fill in your answer in the following table.

	Zero	De-normalized	Floating point	infinity	NaN
Exponent	0	0	1-2046	2047	2047
Significand	0	nonzero	anything	0	nonzero

非正規數：

逐漸下溢(gradual underflow)：(IEEE 754 有規範，但 MIPS 並沒有)指數之無號數值為 0，但有效數字不為 0，並容許在精確度上逐漸遞減直到變成 0 為止。例如：最小的單精度浮點數是 $1.0000\ 0000\ 0000\ 0000\ 0000\ 000\ 0 \times 2^{-126}$
但非正規則為： $0.0000\ 0000\ 0000\ 0000\ 0000\ 001\ 0 \times 2^{-126} = 1.0 \times 2^{-149}$

例(3) : Given the IEEE 754 single precision floating-point operand format as follows,

Sign(s)	Exponent(e)	Fraction(f)
1	8	23

1. The decimal number -13.5 is represented as: 1 1000 0011 1011 0000 0000 0000 000
2. The largest positive number normalized number is $(1-2^{-23}) * 2^{+127}$
3. The smallest positive de-normalized number is $+ 2^{-149}$
4. Exponents with all 1's are reserved for +-無限 and NaN

3、4

註(1) : $-13.5 = -1101.1 = -1.1011 * 2^3$

⇒ 1 1000 0010 1011 0000 0000 0000 000

註(2) : The largest positive number = $(1+1-2^{-23}) * 2^{127} = (1-2^{-24}) * 2^{128}$

例(23) : Multiple choices – please circle all correct answers. Consider the IEEE 754 single precision floating-point (FP) format. Which of the following statements are correct?

1. The FP number representation is $(-1)^{\text{sign}} * (\text{significand}) * 2^{\text{exponent-bias}}$
2. The smallest positive normalized number is: $1.0 * 2^{-149}$
3. The smallest de-normalized number is: $1.0 * 2^{-149}$
4. Converting an integer variable to a single precision FP number will lose precision.

1、3、4 ; 註 : *significand = 1.Fraction*

例(9) : Assume we are designing a 16-bit MIPS CPU with 16-bit instruction words.

1. Assume the IEEE-754 floating-point representation is also adjusted to 16-bit long for this 16-bit MIPS CPU. If the floating-point numbers are required to represent the values within $\pm 10^{18}$, what are the numbers of bits of the field in this 16-bit floating-point format? Please explain your reasons, (assume $\log_{10} 2 = 0.3$)
2. In the IEEE-754 floating-point representation, the exponent field employs the excess-N coding. What should be N in your 16-bit floating-point format if it follows the similar mechanism of IEEE 32-bit standard?

1. 表格如下

<i>a</i>	<i>b</i>	<i>C</i>
<i>sign</i>	<i>exponent</i>	<i>Mantissa</i>
1	6	9

2. $N = 2^{6-1} - 1 = 31$

註 : $10^{18} = 2^{18/0.3} = 2^{60} = 2 * 2^{59}$. If size of exponent field is 7 bits, the maximum positive exponent will be $63 > 59$ and if the size of exponent field is 6 bits, the maximum positive exponent will be $31 < 59$

練習 : 假設有一個 16 位元 IEEE 754 浮點數的格式，其指數佔 5 個位元。以下哪一個最可能是它可以表示的數字範圍？

1. $1.0000\ 0000\ 00 * 2^0$ to $1.1111\ 1111\ 11 * 2^{31}$, 0
2. $\pm 1.0000\ 0000\ 0 * 2^{-14}$ to $\pm 1.1111\ 1111\ 1 * 2^{15}$, ± 0 , $\pm$ 無限, NaN
3. $\pm 1.0000\ 0000\ 00 * 2^{-14}$ to $\pm 1.1111\ 1111\ 11 * 2^{15}$, ± 0 , $\pm$ 無限, NaN
4. $\pm 1.0000\ 0000\ 00 * 2^{-15}$ to $\pm 1.1111\ 1111\ 11 * 2^{14}$, ± 0 , $\pm$ 無限, NaN

3

例(15) : (Choose all right statements) Floating-point format

1. Suppose there was a 16-bit IEEE 754 floating-point format with 5 exponent bits. The likely range of numbers it could represent would be $\pm 1.0000\ 0000\ 00 * 2^{-14}$ to $1.1111\ 1111\ 11 * 2^{15}$, ± 0 , $\pm$ 無限, NaN

- For 32-bit IEEE 754 floating-point standard, the smallest positive single precision normalized number is: $1.0000\ 0000\ 0000\ 0000\ 0000\ 000\ * 2^{-125}$
- For 32-bit IEEE 754 floating-point standard, the smallest single precision de-normalized number is: $0.0000\ 0000\ 0000\ 0000\ 0000\ 001\ * 2^{-126}$
- IEEE-754 floating-point standard adopts bias scheme to increase maximum number in exponent part.

1、3

註(3) : The answer should be $-0.1111\dots1 * 2^{-126}$, but this subject also appear in 2010 and the announced answer by NCTU is $0.0000\dots1 * 2^{-126}$. So, C should be chosen in this place.

練習 : List four floating-point operations that cause NaN to be created?

- Divide 0 by 0
- Multiply 0 by infinity
- Any floating point operation involving NaN
- Adding infinity to negative infinity

練習 : Assuming single precision IEEE 754 format, what decimal number is represented by this word:

1 01111 1101 0010 0000 0000 0000 0000 000

$$-1.001 * 2^{-2} = -0.28125$$

練習 : Using 32-bit IEEE 754 single precision floating point with one (1) sign bit, eight (8) exponent bits and twenty-three (23) mantissa bits, show the representation of -11/16 (-0.6875)

1 0111 1110 0110 0000 0000 0000 0000 000

練習 : What is the smallest positive (not including +0) representable number in 32-bit IEEE 754 single precision floating point? Show the bit encoding and the value in base 10 (fraction or decimal OK).

$$0\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 001 = 1.0 * 2^{-149}$$

練習 : Define the WiMPY precision IEEE 754 floating point format to be:

X XXX XXXX

Where each X represents one bit. Convert each of the following WiMPY floating point numbers to decimal: 1、0000 0000 ; 2、1101 1010 ; 3、0111 0000

- +0
- $-1.101 * 2^2 = -6.5$
- +無限

練習 : For the decimal numbers -938.8125

- Write down the binary representation of the decimal number, assuming the IEEE 754 single precision format.
- Write down the binary representation of the decimal number, assuming it was stored using the single precision IBM format (base 16, instead of base 2, with 7 bits of exponent).

$$1. \quad -938.8125 * 10^0 = -11\ 010\ 010.1101 * 2^9 = 1.1101010101101 * 2^9 \\ \Rightarrow 1\ 1000\ 1000\ 1101\ 0101\ 0110\ 1000\ 0000\ 000$$

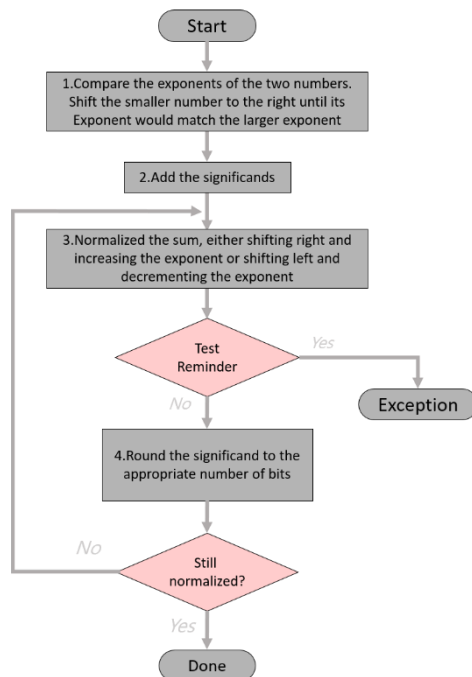
$$\begin{aligned}
2. \quad & -938.8125 * 10^0 = -11\ 010\ 010.1101 * 2^0 = -3AAD.D * 16^0 \\
& = -0.3AAD * 16^3 = -0.001\ 1010\ 1010\ 1101 * 16^3 \\
& \Rightarrow 1\ 1000\ 011\ 0011\ 1010\ 1010\ 1101\ 0000\ 0000
\end{aligned}$$

練習：The IEEE 754 floating-point standard specifies 64-bit double precision with a 53-bit significand (including the implied 1) and an 11-bit exponent. IA-32 offers an extended precision option with a 64-bit significand and a 16-bit exponent.

1. Assuming extended precision is similar to single and double precision, what is the bias in the exponent?
2. What is the range of numbers that can be represented by the extended precision option?
3. How much greater is this accuracy compared to double precision?

1. $2^{15} - 1 = 32767$
2. As small as $2.0 * 10^{-9864}$, almost as large as $2.0 * 10^{9864}$ ($1.0 * 2^{-32766}$ to $1.11...1 * 2^{+32726}$)
3. $(64-53)/53 = 20\%$ more significant digits

重點十二：浮點數加法



1. 將指數較小的數的有效數字部分向右移位，直到它的指數與較大的數的指數部分一樣為止
2. 將有效數字相加
3. 將總和正規化，並檢查是否有溢位
4. 將結果四捨五入

例：

FP Addition : $0.5 + (-0.4375) = ?$

$$0.5 = 1.000 * 2^{-1}$$

$$-0.4375 = -1.110 * 2^{-2}$$

$$1. -1.110 * 2^{-2} = -0.111 * 2^{-1}$$

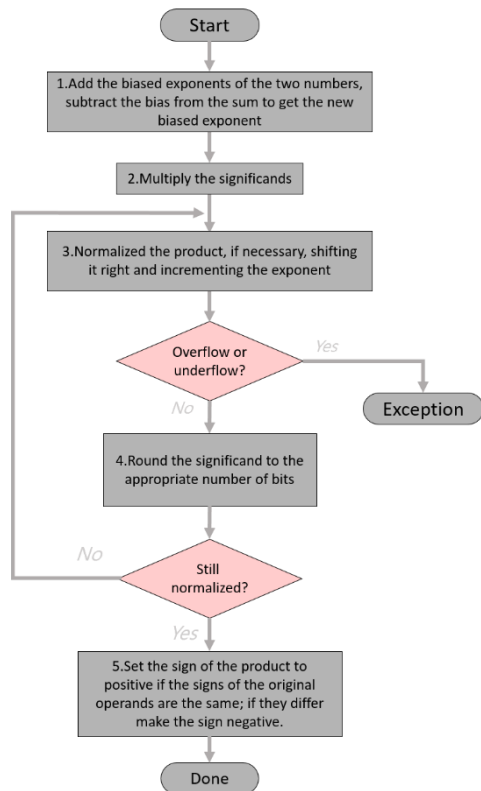
$$2. (1.000 - 0.111) * 2^{-1} = 0.001 * 2^{-1}$$

$$3. 0.001 * 2^{-1} = 1.000 * 2^{-4}$$

$$4. 1.000 * 2^{-4} = 1.000 * 2^{-4}$$

$$1.000 * 2^{-4} = 0.0001 = 0.0625$$

重點十三：浮點數乘法



1. 將兩偏移指數相加計算乘積的指數 (注意：偏移值必須再扣除一次)。
2. 將兩個有效數字相乘。
3. 正規化乘積並應檢查指數是否會導致溢位或下溢。
4. 四捨五入乘積。若四捨五入的結果會讓有效數字變成非正規格式，則再正規化一次，並且再檢查指數的大小
5. 設定乘積正負號：如果兩運算元異號則乘積符號位元為 1，否則為 0

例：

FP Multiplication

FP Addition : $0.5 * (-0.4375) = ?$

$$0.5 = 1.000 * 2^{-1}$$

$$-0.4375 = -1.110 * 2^{-2}$$

$$1. -1 + (-2) = -3 \quad (126+125) - 127 = 124$$

$$2. (1.000 * 0.111) * 2^{-3} = 1.110 * 2^{-3}$$

$$3. 1.110 * 2^{-3} = 1.110 * 2^{-3}$$

$$4. 1.110 * 2^{-3} = 1.110 * 2^{-3}$$

$$5. -1.110 * 2^{-3}$$

$$-1.110 * 2^{-3} = -0.21875$$

例(73)：

1. Given the flow diagram of the procedures for multiplying two binary floating-point numbers.
2. Multiply the two decimal numbers 0.75 and -0.375 by using the steps from your answer in 1. Show the step-by-step intermediate results in your answer.

1. 如圖

2. In binary, the task is $1.100 * 2^{-1} * 1.100 * 2^{-2}$

$$1. -1 + (-2) = -3 \quad (126+125) - 127 = 124$$

$$2. (1.100 * 1.100) * 2^{-3} = 10.010000 * 2^{-3} = 10.01 * 2^{-3}$$

$$3. 10.01 * 2^{-3} = 1.001 * 2^{-2}$$

$$4. 1.001 * 2^{-2} = 1.001 * 2^{-2}$$

$$5. -1.001 * 2^{-2}$$

$$\Rightarrow 1.001 * 2^{-2} = -0.01001 = -0.2812$$

重點十四：精確的算術運算

四捨五入看似簡單，但實際上需要額外的硬體支援才可執行。IEEE 754 規範運算中的數值有多保留 2 個額外位元，分別稱為保護位元(guard bit)與捨入位元(round bit)，可簡寫為 GR，若 GR=0~49 則捨去；若 GR=51~99 則進位

練習：假設我們有三位元有效數字，試將 2.56×10^0 與 2.34×10^2 相加並捨至最接近的三位元十進位數。(先計算有保護位元與捨入位元的情形，然後再計算沒有的狀況)

以十進位為例： $2.56 \times 10^0 + 2.34 \times 10^2$

1. with GR

$$\begin{array}{r} 2.56 \times 10^0 \rightarrow 0.02 \times 10^2 \\ 0.02 \times 10^2 \\ + \quad 2.34 \times 10^2 \\ \hline 2.36 \times 10^2 \end{array}$$

2. without GR

$$\begin{array}{r} 2.56 \times 10^0 \rightarrow 0.0256 \times 10^2 \\ 0.0256 \quad *10^2 \\ + \quad 2.34 \quad *10^2 \\ \hline 2.3656 \quad *10^2 \rightarrow 2.37 \times 10^2 \end{array}$$

例(72)：Add 6.24×10^1 to 9.51×10^2 , assuming that you have only three significant decimal digits. Round to the nearest decimal number, first with guard and round digits and then without them. Explain your work step by step

With guard and round digits:

$$\begin{array}{r} 6.24 \times 10^1 \rightarrow 0.624 \times 10^2 \\ 0.624 \times 10^2 \\ + \quad 9.51 \times 10^2 \\ \hline 10.134 \times 10^2 \rightarrow 10.1 \times 10^2 \rightarrow 1.01 \times 10^3 \end{array}$$

Without guard and round digits:

$$\begin{array}{r} 6.24 \times 10^1 \rightarrow 0.62 \times 10^2 \\ 0.62 \times 10^2 \\ + \quad 9.51 \times 10^2 \\ \hline 10.13 \times 10^2 \rightarrow 10.1 \times 10^2 \rightarrow 1.01 \times 10^3 \end{array}$$

Round to Nearest even (unbiased rounding)：因為偶數較吉利，故有捨、進至靠近的偶數傳統

1. $0.2750 \Rightarrow 0.28$

2. $0.2650 \Rightarrow 0.26$

ULP：捨去的位元數(必為整數值)，例如： $2.34 \times 10^5 + 2.56 \times 10^0$

$$\begin{array}{r} 2.56 \times 10^0 \rightarrow 0.0000256 \times 10^5 \\ 0.0000256 \times 10^5 \\ + \quad 2.34 \times 10^5 \\ \hline 2.3400256 \times 10^5 \rightarrow 2.34 \times 10^5 \Rightarrow \text{故 ULP}=5 \end{array}$$

例(22) : Which of the following statements are true?

- In floating point number, the binary point is not fixed.
- Units in the last place (ulp) is the number of bits in error in the most significant bits of the significand between the actual number and the number that can be represented.
- When a negative exponent of a floating point number becomes too large to fit in the exponent field, underflow occurs.
- In floating-point addition, before addition of the significands, the decimal point of the number that has the smaller exponent should be aligned.
- None of tire above

a、c、d ; 註(b) : Units in the last place (ulp) is the number of bits in error in the least significant bits of the significand between the actual number and the number that can be represented.

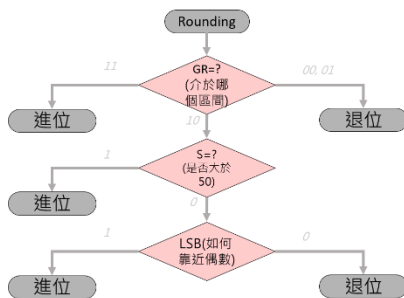
Sticky bit

在十進位中，GR 在 0~49 與 51~99 兩區間都沒有執行上的問題，唯有 50 的時候會需要考慮是否進位或捨入。因此 IEEE 754 多加入一個黏著位元(Sticky bit)，若是 R 之後仍有數值但被捨去，Sticky bit 便會設定為 1，故當 GR=50 的狀況發生時，可以透過 S 來判斷是否大於 50，若是則進位、若否則再處理(依 LSB 決定如何執行以靠近偶數)

練習：假設在上例中我們是把 5.01×10^{-1} 和 2.34×10^2 相加，分別計算有黏著位元與沒有黏著位元的情形。

- 有保護位元與捨入位元，因此總和為 2.3450
- 由於在右邊有非零數字，所以黏著位元必須被設定。若沒有黏著位元來記錄是否有任何 1 被截掉了，我們會假設這個數字等同於 2.3450000...0，並將它捨入至最近的偶數 2.34
- 若有黏著位元來輔助記憶這個數比 2.345000.0 大，則我們會將它進位至 2.35

綜合上述關於 G、R、S、LSB，我們可以得到下面的判斷圖：



例(11) : (Choose all right statements) Assume we have three significant decimal digits, and the accuracy of a floating number is to the second decimal place. Round the value of $2.85 \times 10^4 + 5.16 \times 10^6$ to the nearest decimal number with three significant decimal digits, first with guard and round digits (V_GR), and then without them (V_NoGR).

- The value of V_GR is 5.19×10^6 .
- The value of V_NoGR is 5.17×10^6 .
- The ulp of V_GR is 0.15.
- Sticky bit is another scheme to improve accuracy by setting it to the value of performing Boolean XOR operation on all bits to the right of the round bit.

1 ; 註(2) : V_{NoGR} is 5.18×10^6 ; 註(3) : $ulp=2$; 註(4) : Sticky bit is set whenever there are nonzero bits to the right of the round bit

練習 : The floating-point format to be used in this problem is an 8-bit IEEE 754 normalized format with 1 sign bit, 4 exponent bits, and 3 mantissa bits. It is identical to the 32-bit and 64-bit formats in terms of the meaning of fields and special encodings. The exponent field employs an excess-7 coding. The bit fields in a number are (sign, exponent, mantissa). Assume that we use unbiased rounding to the nearest even specified in the IEEE floating point standard.

1. Encode the following numbers the 8-bit IEEE format: 1. 0.0011 011 ; 2. 16.0
2. Perform the computation $1.011 + 0.0011\ 011$ showing the correct state of the guard, round and sticky bits. There are three mantissa bits.
3. Decode the following 8-bit IEEE number into their decimal value: 1 1010 101
4. Decide which number in the following pairs are greater in value (the numbers are in 8-bit IEEE 754 format): 1. 0100 100 and 0 0100111 ; 2. 1 1100 100 and 1 1100 101
5. In the 32-bit IEEE format, what is the encoding for negative zero?
6. In the 32-bit IEEE format, what is the encoding for positive infinity?

1. $0.0011\ 011 = 1.1011 \times 2^{-3} \rightarrow 0\ 0100\ 110$ (after applying round to the nearest even)
 $16.0 = 10000 = 1.00 \times 2^4 = 0\ 1011\ 000$

2. 算式如下 :

1.01100 最右邊兩位元為 guard 跟 round
+0.00110 sticky bit 設為 1

1.10010

After rounding, the answer then is (1.101)

3. The decimal value is $-2^3 \times 1.62 = -13.0$
4. The first number = $2^{-3} \times 1.5 = 0.125 \times 1.5 = 0.1875$
The second number = $2^{-3} \times 1.875 = 0.234375$
The second number is greater in value

The first number = $-2^5 \times 1.1 = -48.0$

The second number = $-2^5 \times (1.101) = -52.0$

The first number is greater in value.

5. The representation of negative zero in 32-bit IEEE format is
1 0000 0000 0000 0000 0000 0000 000
6. The representation of positive infinity in 32-bit IEEE format is
0 1111 1111 0000 0000 0000 0000 000

例(40) : Assume the floating-point format to be used in this problem is an 8-bit IEEE 754 normalized format with 1 sign bit, 4 exponent bits, and 3 mantissa bits. It is identical to the 32-bit and 64-bit formats in terms of meaning of fields and special encodings. The bit fields in a number are (sign, exponent, mantissa). Assume we use unbiased rounding to the nearest even specified in the IEEE floating point standard. Copy the following table into the answer sheet and fill in your answers for each.

1. The exponent field employs an excess-N coding. What should be N if it follows the IEEE 32-bit standard?
2. Please encode the binary number, 0.0011 011, using this 8-bit IEEE format. Apply rounding if necessary.
3. What decimal value does this binary, 1101 0101, represent?

1	2	3
7	0010 0110	-13

練習：The floating-point format to be used in this problem is a normalized format with 1 sign bit, 3 exponent bits, and 4 mantissa bits. The exponent field employs an excess-3 coding. The bit fields in a number are (sign, exponent, mantissa). Assume that we use unbiased rounding to the nearest even specified in the IEEE floating point standard.

1. Encode the following numbers in the above format: 1 ∼ 1.0 ; 2 ∼ 0.0011 011
 2. In one sentence for each, state the purpose of guard, rounding, and sticky bits for floating point arithmetic.
 3. Perform rounding on the following fractional binary numbers, use the bit positions in italics to determine rounding (use the rightmost 3 bits):
 1. Round to positive infinity: +0.1001 0111 0
 2. Round to negative infinity: -0.0011 1100 1
 3. Unbiased to the nearest even: +0.1001 0110 0
 4. Unbiased to the nearest even: -0.1001 0011 0
 4. What is the result of the square root of a negative number?
1. *The encoding for the above number is 0 011 000 Underflow*
 2. *The guard bit is an extra bit that is added at the least significant bit position during an arithmetic operation to prevent loss of precision.
The round bit is the second bit that is used during an arithmetic operation to prevent loss of precision.
The sticky bit keeps record of any 1's that have been shifted on to the right beyond the guard and round bits*
 3. *如下：*
 1. *Round to positive infinity: +0.1001 0111 0 = +0.1001 10*
 2. *Round to negative infinity: -0.0011 1100 1 = -0.0100 00*
 3. *Unbiased to the nearest even: +0.1001 0110 0 = 0.1001 10*
 4. *Unbiased to the nearest even: -0.1001 0011 0 = -0.1001 01*
 4. *In the IEEE 754 standard, the square root of negative number results in an exception being generated by the floating point arithmetic unit.*

練習：NAVIDIA has a 'half' format, which is similar to IEEE 754 except that it is only 16 bits wide. The leftmost bit is still the sign bit, the exponent is 5 bits wide and stored in excess-15 format, and the mantissa is 10 bits long. A hidden 1 is assumed. The following table shows pairs of decimal numbers. Calculate the sum of A and B by hand, assuming A and B are stored in the 16-bit NVIDIA format. Assume one guard, one round bit and one sticky bit, and round to the nearest even. Show all the steps.

a	-1.278*10 ³	-3.90625*10 ⁻¹
b	2.3109375*10 ¹	6.391601562*10 ⁻¹

a.	$-1.278 \times 10^3 = -1278 = -1001\ 1111\ 110 = -1.0011\ 1111\ 10 \times 2^{10}$ $-3.90625 \times 10^{-1} = -0.390625 = -1.1001\ 0000\ 00 \times 2^{-2}$ <i>Shift binary point 12 to the left to align exponents,</i> $-1.1001\ 0000\ 00 \times 2^{-2} \rightarrow -0.0000\ 0000\ 0001\ 1 \times 2^{10}$ GR $-1.0011\ 1111\ 10\ 00$ $-0.0000\ 0000\ 00\ 01\ 1$ (Guard=0, Round=1, Sticky=1) $-1.0011\ 1111\ 10\ 01$ Guard=0, Round=1, Sticky=1, Round down. $-1.0011\ 1111\ 10 \times 2^{10} = -1.278 \times 10^3$
b.	$2.3109375 \times 10^1 = 23.109375 = 1.0111\ 0001\ 11 \times 2^4$ $6.391601562 \times 10^{-1} = .6391601562 = 1.0100\ 0111\ 01 \times 2^{-1}$ <i>Shift binary point 5 to the left to align exponents,</i>

	GR
	+1.0111 0001 11 00
	+0.0000 1010 00 11 101 (<i>Guard=1, Round=1, Sticky=1</i>)
	+1.0111 1011 11 11
	<i>In this case Guard and Round are both 1, so we round up.</i>
	+1.0111 1100 00*2 ⁴ = 1 0111.1100 0*2 ⁰ = 23.75 = 2.375*10 ¹

練習：The following table shows pairs of decimal numbers.

a.	5.66015625*10 ⁰	8.59375*10 ⁰
b.	6.18*10 ²	5.796875*10 ¹

Calculate the product of A and B by hand, assuming A and B are stored in the 16-bit NVIDIA format. Assume one guard, one round bit and one sticky bit, and round to the nearest even.

a.	$5.66015625 = 1.0110\ 1010\ 01*2^2$ $8.59375 = 1.0001\ 0011\ 00*2^3$ <i>Exp: 2+3 = 5, 5+15 = 20 (10100)</i> <i>Mantissa:</i> $\begin{array}{r} 1.0110\ 1010\ 01 \\ * \quad 1.0001\ 0011\ 00 \\ \hline 1.1000\ 0101\ 0010\ 0010\ 1100 \\ 1.1000\ 0101\ 00\ \underline{10}\ 0010\ 1100\ \textit{Guard=1, Round=0, Sticky=1: Round up} \\ 1.1000\ 0101\ 01*2^5 = 0\ 10100\ 1000\ 0101\ 01 \end{array}$
b.	$6.18*10^2 = 618 = 1.0011\ 0101\ 00*2^9$ $5.796875*10^1 = 57.96875 = 1.1100\ 1111\ 11*2^5$ <i>Exp: 9+5 = 14, 15+14 = 29 (11101)</i> <i>Signs: both positive, result positive</i> <i>Mantissa:</i> $\begin{array}{r} 1.0011\ 0101\ 00 \\ * \quad 1.1100\ 1111\ 11 \\ \hline 10.0010\ 1111\ 1100\ 0010\ 1100 \\ \textit{Must Normalize, add one to exponent} \\ 1.0001\ 0111\ 11\ \underline{10}\ 0001\ 0110\ 0\ \textit{Guard=1, Round=0, Sticky=1: Round up} \\ 1.0001\ 1000\ 00*2^{15} = 0\ 1110\ 0001\ 1000\ 0 \end{array}$

練習：The following table shows fractions to be represented in various floating-point formats.

a.	1/2
b.	1/9

- Write down the bit pattern in the mantissa assuming a floating-point format that uses binary numbers in the mantissa. Assume there are 24 bits, and you do not need to normalize. Is this representation exact?
- Write down the bit pattern in the mantissa assuming a floating-point format that uses Binary Coded Decimal (base 10) numbers in the mantissa instead of base 2. Assume there are 24 bits, and you do not need to normalize. Is this representation exact?
- Write down the bit pattern assuming that we are using base 15 numbers in the mantissa instead of base 2. (Base 16 numbers use the symbols 0-9 and A-F. Base 15 numbers would use 0-9 and A-E) Assume there are 24 bits, and you do not need to normalize. Is this representation exact?
- Write down the bit pattern assuming that we are using base 30 numbers in the mantissa instead of base 2. (Base 30 numbers would use 0-9 and A-T) Assume there are 20 bits, and you do not need to normalize.

1	a	1000 0000 0000 0000 0000 0000	0x800000	Yes
	b	0001 1100 0111 0001 1100 0111	.1C71C7	No
2	a	0101 0000 0000 0000 0000 0000	.50000	Yes
	b	0001 0001 0001 0001 0001 0001	.111111	No
3	a	0111 0111 0111 0111 0111 0111	.777777	No
	b	0001 1010 0000 0000 0000 0000	.1A0000	Yes
4	a	01111 00000 00000 00000	.F000	Yes
	b	00011 01010 00000 00000	.3A00	Yes

重點十五：浮點數加法的結合律

浮點數加法並不滿足結合律

例(51)：For floating operations, (A+B)+C is equal to A+(B+C).

Floating point addition does not possess associative.

假設 $A = -1.5 \times 10^{38}$; $B = 1.5 \times 10^{38}$; $C = 1.0$

$(A+B)+C = 0.0$ but : $A+(B+C) = 1.0$

$(A+B)+C$ is not equal to $A+(B+C)$.

重點十六：右移與 2 的冪次方除法運算

負數直接向右移並不會像正數一樣，可能會出錯。例：

$-5(1011) / 4 = -1$ ，但向右移會變成： $-2(1110)$

$-8(1000) / 4 = -2(1110)$

原因在於：負數的表示法為負的 MSB 去加上剩餘的數值，若直接右移等於刪去相加的數，故會出現錯誤

5 除 4 右移結果： $(10.11) = -2^1 + 2^{-1} + 2^{-2} = -2 + 0.5 + 0.25 = -2 + 0.75 = -1.25$

由上例可知，修正方式：右移幾個 bits，就先加上幾個 bits 1 的數值

$-5/4 \Rightarrow 1011 + 11 = 1110 \Rightarrow 11.10 = (11)11 = -1$

$-8/4 \Rightarrow 1000 + 11 = 1011 \Rightarrow 10.11 = (11)10 = -2$

例(62)：In coding assembly, an integer multiply by a power of 2 can be replaced by a left shift, and an integer division by a power of 2 can be replaced by a right shift. (改正題)

右移指令會等同於 2 之冪次方的整數除法，這在無號數中是正確的，但在有號數中卻是錯誤的。例如 $-5/4$ 的商數應該是 -1 ，但直接右移的結果卻是 -2 而非 -1

例(25)：(Circle all correct answers) I is an integer variable. Which of the following expressions give the correct result of $(I/8)$?

1. $(I+7) >> 3$
2. $(I>0? I>>3: (I+7)>>3)$
3. $I >> 3$
4. $(I + (I>>31) \& 7) >> 3$

2、4；註(4)：若為正，則直接刪去 3 個 bits；若為負，先加上 3 個 bits 的 111，再右移

重點十七：軟體偵測溢位

Signed	Unsigned
addu \$t0, \$t1, \$t2	addu \$t0, \$t1, \$t2
xor \$t3, \$t1, \$t2	nor \$t3, \$t1, \$zero
slt \$t3, \$t3, \$zero	sltu \$t3, \$t3, \$t2
bne \$t3, \$zero, Non-overflow	bne \$t3, \$zero, overflow
//是否異號	
xor \$t3, \$t0, \$t1	
slt \$t3, \$t3, \$zero	
bne \$t3, \$zero, Overflow	
//測試結果	

例(39)：Consider a sequence of MIPS instructions that can be used to discover overflow when adding two signed integers stored in registers \$t1 and \$t2. In this sequence, A and C denote two instructions names and B denotes an operand. What are A, B, and C, respectively? (Note: instruction slt performs set-on-less-than operation). (Mark you answer clearly)

```

addu    $t0, $t1, $t2
A        $t3, $t1, $t2
slt      $t3, $t3, $zero
C        $t3, B, Non-overflow
A        $t3, $t0, $t1
slt      $t3, $t3, $zero
C        $t3, B, Overflow

```

A: xor 、 B: \$zero 、 C: bne

例(14)：Consider two code sequences for detecting the overflow of addition on two numbers in registers \$t1 and \$t2. One addition is signed addition while the other is unsigned addition.

- The code sequence in bottom left figure is to detect signed addition overflow. Explain how It can detect signed addition overflow.
- The code sequence in the bottom right figure is to detect unsigned addition overflow. Explain how it can detect unsigned addition overflow.

(You do not need to describe the meaning of each instruction. The main design concept inside the codes is enough)

addu \$t0, \$t1, \$t2	addu \$t0, \$t1, \$t2
xor \$t3, \$t1, \$t2	nor \$t3, \$t1, \$zero
slt \$t3, \$t3, \$zero	sltu \$t3, \$t3, \$t2
bne \$t3, \$zero, Non-overflow	bne \$t3, \$zero, overflow
xor \$t3, \$t0, \$t1	
slt \$t3, \$t3, \$zero	
bne \$t3, \$zero, Overflow	

- 兩個異號數相加不會發生溢位，而兩個正數相加變負數，或兩個負數相加變正數便會發生溢位。
- $\$t1 + \$t2 > 2^{32}-1 \rightarrow \text{overflow}$
 $\$t2 > 2^{32}-1 - \$t1 \rightarrow \text{overflow}$
 $\$t2 > \$t1 \rightarrow \text{overflow}$

練習：Find the shortest sequence of MIPS instructions to determine there is a carry out from the addition of two registers, say, register \$t3 and \$t4. Place a 0 or 1 in register \$t2 if the carry out is 0 or 1, respectively. (Hint: It can be done in two instructions.)

```
addu $t2, $t3, $t4
sltu $t2, $t2, $t4
```

Fused Multiply Add(乘加指令)

許多數位訊號處理器都有乘加指令：`mul-add $r1, $r2, $r3 => $r1=$r1+$r2*$r3`
因為乘完再加的精確度與效率不如乘加指令(1 次四捨五入好過 2 次)。

Memory direct:

\$s1=10 \$s2=96 位址 100 存 200 位址 200 存 50

`add $s0, $s1, 4($s2)`

⇒ $96(\$s2)+4 = 100$ ，到位址 100 找到數值 200，再加上數值 10(\$s1)，故 \$s0 等於 210

Memory indirect:

⇒ $96(\$s2)+4=100$ ，到位址 100 找，但因為內容存的是位址，故要再到位址 200 去找到數值 50，再加上數值 10(\$s1)，故 \$s0 等於 60

例(第一章 45)：How many memory accesses are required to execute a “store” instruction with memory indirect addressing?

4；註：若 *instruction fetch* 算入 *memory access* 次數，則答案為 4；若不則答案為 3

`Store $s0, 4($s1)`

1.指令、2.找到\$s0 與其值、3.找到\$s1+4 位址、4.將\$s0 之值放入\$s1 +4 的內容地址